

Image classification and segmentation using YOLO

Narayana Darapaneni ¹ Anwesh Reddy Sudha B G

Rohan Gurubhaiye Dinesh Kumar D Gopal S

Radha Acharya Vikram P G Vinod Dinnimani

January 27, 2023

¹Corresponding author

Contents

1	Introduction	10
1.1	Background	10
1.2	Evolution of YOLO	10
1.2.1	Architecture	12
2	Literature Review	14
2.1	Significant Yolo Architectures:	14
2.1.1	YOLOv3 : An Incremental Improvement	14
2.1.1.1	Backbone : Feature Extractor Network	15
2.1.1.2	Neck : Multi-scale Feature Pyramid Layer	15
2.1.1.3	Head : Predictions across 3 different scale	16
2.1.1.4	Loss Function : Binary Cross Entropy loss	16
2.1.2	Tiny-YOLOv3: one-stage enhanced model for Object-Detection and Object-Recognition	17
2.1.2.1	Introduction:	17
2.1.2.2	Architecture:	17
2.1.3	Tinier-YOLO : Application of A real-time Object detection method in a constrained environment	18
2.1.3.1	Introduction:	18
2.1.3.2	Architecture:	19
2.1.4	YOLOv4	20
2.1.4.1	Introduction:	20
2.1.4.2	Architecture:	21
2.1.5	YOLOv5	23
2.1.5.1	Introduction:	23
2.1.5.2	Yolo V 5 Architecture:	23
2.1.5.3	Loss Functions used in YOLO5:	24
2.1.5.4	Binary Cross Entropy in YOLO5:	24
2.1.5.5	Logits in YOLO5:	25
2.1.5.6	Improved Loss Functions for Imbalanced Data	25

2.1.5.7	Balanced Focal Loss	25
2.1.5.8	Loss Rank Mining	26
2.1.5.9	Experiments	26
2.1.5.10	Conclusion	27
2.1.5.11	Advantages and Disadvantages of YOLO V5:	27
2.1.6	YOLO FIR and YOLO FIRI	27
2.1.6.1	Introduction:	27
2.1.6.2	Network Architecture:	28
2.1.6.3	Comparison of the Detection Performance on KAIST infrared pedestrian dataset	29
2.1.7	YOLOv6	30
2.1.7.1	Introduction:	30
2.1.7.2	Architecture of YOLO6 Model:	30
2.1.7.2.1	YOLO6 Backbone Architecture:	30
2.1.7.2.2	YOLO6 Neck Architecture:	30
2.1.7.3	Loss Functions used in YOLO6 Model:	31
2.1.8	YOLOv7	32
2.1.8.0.1	Extended Efficient Layer Aggregation Network (E-ELAN)	34
2.1.8.0.2	Scaling for Concatenation based Mod- els	35
2.1.8.0.3	Rep convolution	35
2.1.8.0.4	Lead and Auxiliary Head	36
2.2	Yolo use cases	37
2.2.1	Object Detection	37
2.2.1.1	Vision Based Intelligent Shelf-Management Sys- tem	37
2.2.1.2	An attention mechanism-improved yolov7 ob- ject detection algorithm for hemp duck count estimation	37
2.2.1.3	A deep learning approach for detection and prediction of kidney diseases	39
2.2.1.4	Multi-Scale Detector for Accurate Vehicle Detection in Traffic Surveillance Data.	39
2.2.1.5	Multi-Agent Deep Reinforcement Learning for Multi-Object Tracker.	40
2.2.1.6	Embedded System Real-Time Vehicle Detec- tion based on Improved YOLO Network.	41
2.2.1.7	A Deep Learning (YOLOv3) Approach for Street Pothole Detection.	41
2.2.2	Pose Estimation	43

2.2.2.1	YOLO-6D+: Single Shot 6D Pose Estimation Using Privileged Silhouette Information	43
2.2.3	Motion detection	43
2.2.3.1	A sport athlete object tracking based on deep sort and yolo v4 in case of camera movement	43
2.2.3.2	Visual object detection and tracking using yolo and sort	44
2.2.4	Human Activity Recognition	44
2.2.4.1	Yolo based human action recognition and localization	44
2.2.4.2	New action recognition frame- work for video highlights summarization in sporting events	44
2.2.4.3	Skeleton based temporal action detection with yolo	44
2.2.4.4	Robot recognizing humans intention and interacting with humans based on a multi-task model combining st-gcn-lstm model and yolo model	45
2.2.4.5	target detection network for dangerous driving gestures	45
2.2.4.6	Abnormal behavior detection in real-time for advanced driver assistance system (adas) using yolo	46
2.2.4.7	Driver drowsiness detection and alert system using yolo	46
2.2.4.8	Human fall detection using yolo: A real-time and ai-on-the-edge perspective	46
2.2.4.9	Safety helmet detection method based on YOLO v4	47
2.2.4.10	A single shot deep pose estimation and analysis	47
3	Foundations	48
3.0.1	Fundamental Architectural Layers	48
3.0.1.1	Convolution Layers :	48
3.0.1.2	Dense Layer	48
3.0.1.3	Deconvolution Layer	49
3.0.1.4	Batch Normalization Layer	49
3.0.1.5	Pooling Layer	49
3.0.1.6	Dropout Layer	50
3.0.1.7	Flatten Layer	50
3.0.1.8	Activation Layer	50

3.0.1.9	Recurrent Layer	51
3.0.2	Activation Functions:	52
3.0.2.1	Sigmoid	52
3.0.2.2	Softmax	53
3.0.2.3	Relu	53
3.0.2.4	Tanh	54
3.0.2.5	LeakyRelu	55
3.0.2.6	ParametricRelu	56
3.0.2.7	RELU6	56
3.0.2.8	SELU	57
3.0.2.9	Swish	57
3.0.2.10	Mish	58
3.0.3	Evaluation Metrics	60
3.0.4	CLASSIFICATION METRICS	60
3.0.4.1	Accuracy	60
3.0.4.2	Precision	61
3.0.4.3	Recall	61
3.0.4.4	F1-score	61
3.0.4.5	AUC-ROC	62
3.0.5	REGRESSION METRICS	63
3.0.5.1	Mean Absolute Error -MAE	63
3.0.5.2	Root Mean Squared Error-RMSE	63
3.0.6	COMPUTER VISION METRICS	63
3.0.6.1	Frames per second (FPS)	63
3.0.6.2	Intersection Over Union	64
3.0.6.3	Mean Average Precision (mAP)	64
3.0.7	Loss Functions	66
3.0.7.1	MSE	66
3.0.7.2	MAE	67
3.0.7.3	Binary Cross-Entropy	67
3.0.7.4	Categorical Cross-Entropy	68
3.0.7.5	Balanced Cross Entropy loss	69
3.0.7.6	Focal loss	69
3.0.7.7	Varifocal loss	70
3.0.7.8	Distribution Focal Loss	71
3.0.8	Optimizers	71
3.0.8.1	SGD	72
3.0.8.2	AdaGrad	73
3.0.8.3	RMSprop	73
3.0.8.4	Adam	74
3.0.8.5	Nesterov Accelerated Gradient Descent	74

3.0.8.6	Adan	75
4	Material and Methods	76
4.0.1	Research Methodology	76
4.0.2	Datasets	76
4.0.2.1	Default Dataset (COCO)	76
4.0.2.2	Custom Dataset	77
4.0.2.3	Environment	77
4.0.3	Data preparation	78
4.0.3.1	Splitting data into training set and validation set	78
4.0.3.2	Preparing the dataset for training	79
4.0.4	Model Building	79
4.0.4.1	YOLO v3	80
4.0.4.2	YOLO v4	82
4.0.4.3	YOLO v7	84
4.0.5	Results analysis	108
5	Conclusion and Future Directions	110

List of Figures

1.1	YOLOv1 Architecture	12
2.1	YOLOv3 : Darknet-53 Backbone Architecture	15
2.2	YOLOv3 : Multi-Scale Feature Pyramid - Neck and Head Architecture	16
2.3	Network architecture of Tiny-YOLOv3.	18
2.4	Performance of YOLOv3 vs Tiny-YOLOv3.	19
2.5	Network architecture of Tinier-YOLO.	20
2.6	Comparison of experiments using Tiny-YOLOv3 and Tinier- YOLO.	20
2.7	YoloV5 Architecture	24
2.8	Binary Cross Entropy Formula	25
2.9	Logits Formula	25
2.10	Improved Loss Formula	26
2.11	Improved Loss Function	26
2.12	Performance Evaluation of Baseline vs Proposed	27
2.13	YOLO FIRI Architecture	28
2.14	YOLO FIRI Performance	29
2.15	YoloV6 Architecture	31
2.16	Formula for Varifocal Loss	31
2.17	YoloV7 Architecture	33
2.18	Extended Efficient Layer Aggregation Network	34
2.19	Model scaling for concatenation-based models	35
2.20	Planned re-parameterized model	35
2.21	Coarse for auxiliary and fine for lead head label assigner.	36
2.22	Vision Based Shelf Life System.	38
2.23	Mosaic Data Augmentation	38
2.24	Multiscale YOLOv3 Object detection network architecture	40
2.25	Yolov3-Live network architecture	41
2.26	Yolov3-Live Performance comparison with YOLOv3-Tiny	42
2.27	Pothole detection using Deep learning	42

3.1	Confusion Matrix	60
3.2	AUC-ROC Curve	62
3.3	Intersection over Union	64
3.4	COCO Metrics	65
3.5	Formula for Varifocal Loss	71

Abstract

This paper aims at a comprehensive study of different versions of YOLO (You Only Look Once) architectures. These network architectures belong to one-stage object detectors, which are further sub-categorized into anchor based. Unlike Region Proposal Network in two-stage networks YOLO uses fixed anchor boxes where in images is divided into grid cells. Such detectors re-frame the classification, localization (regression) into single monolithic regression problem. It visualizes and generalizes well the entire image while training and inferring. While in terms of speed it outperforms its counterparts in two stage detectors, only lags slightly behind in terms of accuracy. Pre-trained models and their weights have been taken from original implementations and setup inference environment and compare resultant metrics of different architectures. The objective evaluated are size (in MBs), performance (FLOPS/FPS) and accuracy (mAP). Finally based on results the study recommend suitable architecture.

Chapter 1

Introduction

1.1 Background

In the last decade, we have made some amazing strides in computer vision, however when it comes to real time application deployment along with accuracy, it's also imperative to focus on memory performance and accuracy.[1] If one can build an application with the help of computer vision, lets say a self-driving vehicle or a robotic control system. For that one need to know what's going on in the image or video. Then it is possible to interact with the physical world in real time. Hence speed of processing becomes extremely important.

In the past, object detection systems would take an image and split it into a bunch of regions. Then run a classifier on each of these regions. High score for that classifier would be considered as detection in the image. While this produced high accuracy of prediction it involved running a classifier thousands of times over an image, thousands of evaluations to produce detection.

1.2 Evolution of YOLO

In 2016 Joseph Redmon[2] introduced present a new approach to object detection. A single network did all detection. It predicted bounding boxes and class probabilities simultaneously. Instead of looking at an image thousands of times to produce detection, with this novel architecture "you only look once". Hence it was called YOLO.

This is one of the architecture belonging to group of Anchor based detectors which intern falls into family of One-Stage detectors. Unlike two-stage detectors, these detectors are not computationally expensive have region proposal stage, instead they divide whole image into grid cells which act as region

proposals. This is done statically hence no computational overhead involved. Hence these networks perform much faster inferences without compromising much on accuracy and efficiency. YOLO re-frames the classification and localization (regression) into a combined monolithic-regression problem. This allows image pixel processing to bounding box coordinates and class probabilities directly. Such monolithic unified model has several advantages.

First, we don't need complex pipeline because YOLO being single-stage anchor-based detector, detection is treated as regression problem. YOLO performs extremely fast at training and inference. Authors claim base network performs inferring at 45 FPS (Frame for each second) on a NVIDIA-Titan-X GPU system and fast version claimed to be performing at 150 fps. This translates to a delay of less than 25 milliseconds of latency while processing a live video streaming.

Second, Unlike two-stage networks using region proposal techniques and sliding window technique in classical computer vision approaches, YOLO visualizes the full image while training and inference. Hence it implicitly encompasses the contextual details/information for the detected objects. Compared to Faster-RCNN, YOLO makes almost less than half mistakes while separating background from class objects.

Third, YOLO having contextual information generalizes well and less probable to fail when applied to new dataset or domain.

While it outperforms compared to its counterparts in two-stage family. It falls short of slightly behind the state-of-art detection system in terms of accuracy. In general early single-stage anchor-based architectures (including YOLO) have challenges with respect to hyper parameter tuning ex. number of anchors, aspect-ratio and size-of-anchor boxes.

Now we looked into some of basic aspects of YOLO, let's dwell into various versions of YOLO and its evolution. The very first-version of YOLOv1 was introduced by Joseph Redmon et [2] all in their paper in the year 2015. It divides input image into $H \times W$ grid. It predicts "B" bounding-boxes, confidence-scores and "C" class-probabilities. This can be represented as $H \times W \times (B * 5 + C)$ tensor. It had inference speed of 45 fps (translates to 22ms per image) and accuracy of 63.4mAP. An experiment that authors claim to be conducted had following parameters $H=W=7$, $B=2$ and $C=20$ as PascalVOC dataset had 20 class labels, the final prediction layer had tensor of $7 \times 7 \times 30$.

1.2.1 Architecture

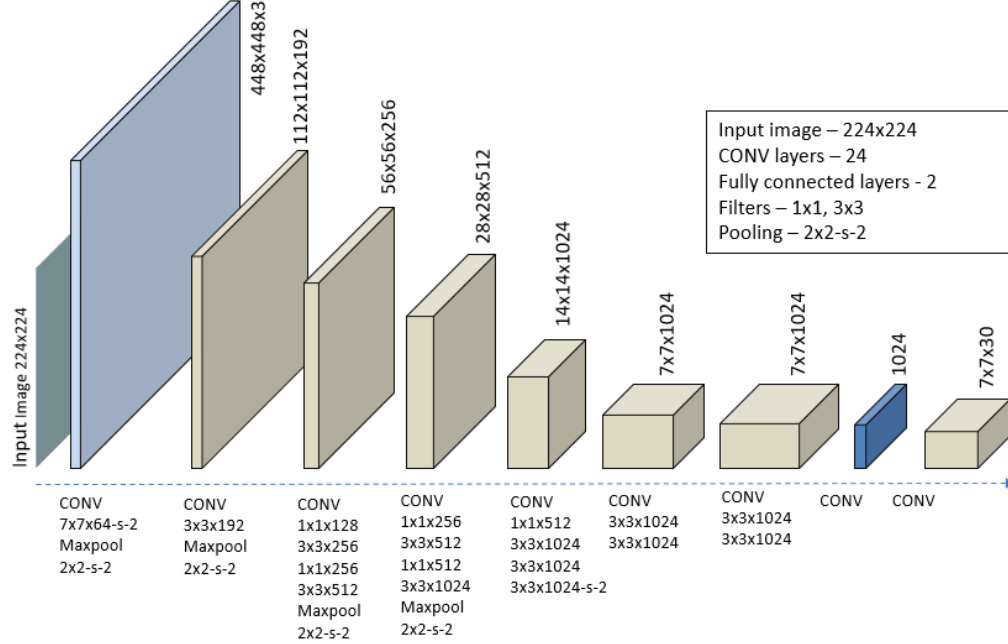


Figure 1.1: YOLOv1 Architecture

The second version YOLOv2 architecture published back then in 2016 by Joseph Redmon and Ali Farhadi in their paper [2]. This is also being called as YOLO9000 as this had over 9000 object categories. The improved YOLOv2 model uses a novel multi-scale training method that allows model to accept various scales of anchors. With this YOLOv2 gets a mAP of 76.8 at 40 fps speed. On ImageNet (44 of 200 classes) dataset a mAP of 19.7 and on COCO (156 classes) dataset mAP of 16.0 was obtained. However YOLOv2 suffers from various limitations compared to available state of the art systems at that time. It uses batch normalization and higher resolution classifier (448x448). Fully connected layers removed from YOLOv1 and one pooling layer is eliminated to make output on CNN layer as 448x448. Input image is scaled to 416x416 to maintain odd number of in feature maps so that single center is obtained. This is good to have when large object tend to occupy center of image instead of multiple locations. With increase of anchor boxes from 98 to more than thousands mAP of 69.2 (69.5 for v1) obtained with a recall of 88%. (81% for v1). Though slight decrease in mAP but significant improvement in recall means model has more room for improvement. Compared to YOLOv1 contrary to short listing priors

or anchor sizes manually, a k-means-clustering used automate finding best priors. A cluster size $k=5$ is selected as good trade-off between complexity and recall. YOLO uses fine grained features. It converts a $26 \times 26 \times 512$ feature-map into $13 \times 13 \times 2048$ feature-map. These feature-maps then concatenated with original features gives an improvement of 1% performance in crease.

The third version YOLOv3 was released by Joseph Redmon and Ali Farhadi in their paper [2] in 2018. This model is little bit bigger than v2 as this has 75 convolution layers. However eliminating some of fully connected layer and pooling layers keeps model size reduced. Performance perspective YOLOv3-320 has 28.2 mAP with inferring speed of 22ms. This has almost same accuracy as v2 but 3 times faster than SSD network. This version borrows concept of feature pyramid network [3] as feature extractor which extracts different size (3) of features then they are concatenated to learn local and global features both. The tensor looks as $N \times N \times [3 * (4+1+80)]$. There are 4 bounding box offsets. There is 1 objectness score. There are 80 class predictions. YOLOv3 comprises of logistic classifier instead of softmax. It has binary-cross-entropy loss for class predictions. YOLOv3 used Darknet53 as backbone architecture with input image size of 256×256 .

The forth version YOLOv4 was published by Alexey Bochkovskiy and Chien-Yao Wang et all in their paper [2] in 2020. Several improvements have been included. The concept of Bag of freebies was introduced, which is about improvements without compromising on inference speed.

1. This uses various data augmentation techniques such as a. CutMix - performs cropping and merging of various other images. It adjust labels according to size of mix area. b. MixUp - multiplies two images. It Then superimpose them with different coefficient ratios and adjust labels accordingly.

2. Various experiments conducted with different type of bounding box regression losses such as mean squared error, Inter section over union loss, CIoU loss, DIoU loss etc. 3. Regularization techniques such as DropOut, DropConnect, Spatial dropout, DropBlock have been used. 4. Normalization techniques such as Cross Mini-Batch Normalization (CmBN) which has proven to increase accuracy. along with this Iteration-batch normalization and GPU normalization have also been used.

Another concept Bag-of-Specials is introduced. This includes plugin modules and post-processing methods. These methods increase accuracy significantly by keeping inference cost to minimal.

Chapter 2

Literature Review

2.1 Significant Yolo Architectures:

2.1.1 YOLOv3 : An Incremental Improvement

The third version YOLOv3 was released by Joseph Redmon and Ali Farhadi in their paper [4] in 2018. This model is little bit bigger than v2 as this has 75 convolution¹ layers. However eliminating some of fully connected layer and pooling² layers keeps model size reduced. Performance perspective YOLOv3-320 has 28.2 mAP³ with inferring speed of 22ms. This has almost same accuracy⁴ as v2 but 3 times faster than SSD network. This version borrows concept of feature pyramid network⁵ as feature extractor which extracts different size (3) of features then they are concatenated to learn local and global features both. The tensor looks as $N \times N \times [3 \times (4 + 1 + 80)]$. This includes four bounding-box offsets, one objectness-score and eighty class predictions. YOLOv3 utilizes logistic classifier instead of softmax⁶ and binary-cross-entropy⁷ loss for class predictions. YOLOv3 used Darknet53 as backbone architecture with input image size of 256x256. Comparison of perfor-

¹Convolution [3.0.1.1](#)

²Pooling [3.0.1.5](#)

³mAP ??

⁴accuracy ??

⁵FPN ??

⁶Softmax [3.0.2.2](#)

⁷Binary-Cross-Entropy [3.0.7.3](#)

Backbone	Top-1	Top-5	Bn Ops	BFLOP/s	FPS
Darknet-19	74.1	91.8	7.29	1246	171
ResNet-101	77.1	93.7	19.7	1039	53
ResNet-152	77.6	93.8	29.4	1090	37
Darknet-53	77.2	93.8	18.7	1457	78

Table 2.1: Comparison of backbones.

mance is show in table.

2.1.1.1 Backbone : Feature Extractor Network

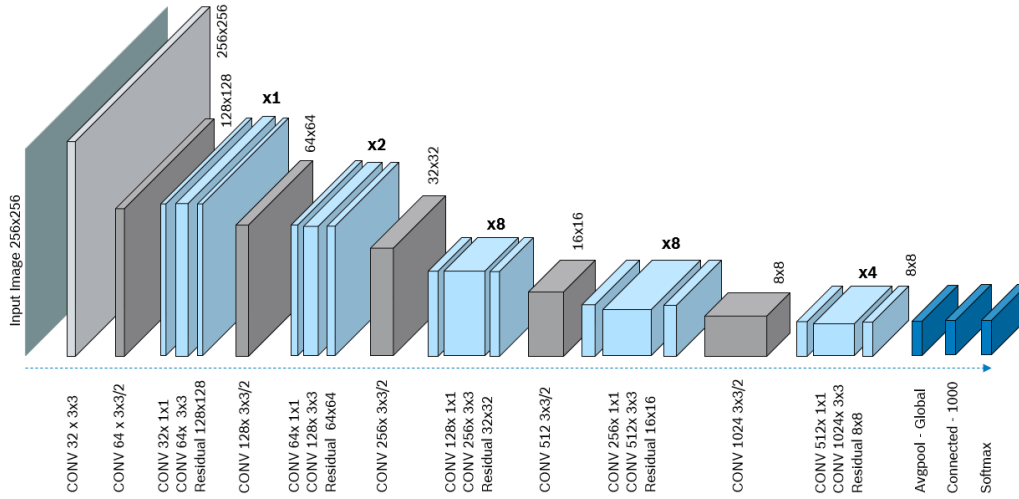


Figure 2.1: YOLOv3 : Darknet-53 Backbone Architecture

In contrast to Yolov2 using Darknet19 as backbone, Yolov3 uses a 53 convolution layer backbone network architecture, authors named it Darknet-53. It uses successive 3x3 and 1x1 convolution filter sizes. This network has greater complexity than Darknet19. However it exhibit efficiency as well compared ResNet101 and ResNet152.

2.1.1.2 Neck : Multi-scale Feature Pyramid Layer

Yolov3 estimates 3 different scale of bounding boxes. The last layers estimate 3 dimensional tensor which encodes bounding-box coordinates, score of objectness and prediction of class probability. Authors take 2x up-sampled feature map and merge them with the residual feature maps from last two feature map layers.

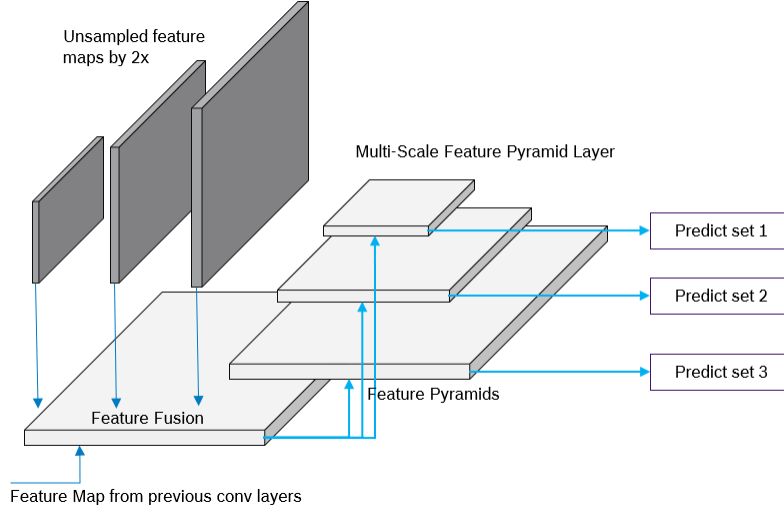


Figure 2.2: YOLOv3 : Multi-Scale Feature Pyramid - Neck and Head Architecture

2.1.1.3 Head : Predictions across 3 different scale

The experiment by author on COCO predicts 2 boxes at each scale, so output tensor looks as follows,

$H \times W \times [3 * (4 + 1 + 80)]$

4 : bounding boxes,

1 : objectness,

80 : classes probabilities,

3 : three Different scale of predictions. (Prediction set 1 to 3).

2.1.1.4 Loss Function : Binary Cross Entropy loss

Authors are not using softmax as they find it not necessary. Binary-Cross-Entropy loss (also known as Log-loss) being used in predictions of classes.

$$BinaryCrossEntropy(LogLoss) = -1/N \sum_{n=i}^N \sum_{n=j}^M y_{ij} * \log(p_{ij})$$

2.1.2 Tiny-YOLOv3: one-stage enhanced model for Object-Detection and Object-Recognition

2.1.2.1 Introduction:

Tiny-YOLOv3[5] is a modified and scaled down version of YOLOv3. As former two-stage model and even single-stage SDD and YOLOv3 are comparatively slower in execution, they are not best suited for application where speed is important than accuracy. So authors Pranav Et al.[5] proposed new architecture Tiny-YOLO which is approx. 442% faster than YOLOv3 and previous versions.

2.1.2.2 Architecture:

Compared to YOLOv3, tiny-YOLO uses Max-pooling layers and reduces features layer by layer hence parameters there by increasing speed. It employs 2 different scales and predicts objectness score, Bounding box coordinates, Class probability score.

Prediction by two different scale produce two different bounding box at 13x13 and 26x26 feature maps. These are concatenated together with a up-sampled feature map of 26x26.

Authors Pranav Et al. [5] evaluated results on PASCAL VOC 2012 datasets with NVIDIA Geforce GTX 1050 GPU with 8GB of RAM. From below graphs it can be concluded that though this has almost 1/4th of accuracy as YOLOv3. YOLOv3-Tiny is much faster and suited for applications where speed is of significance compared to the accuracy.

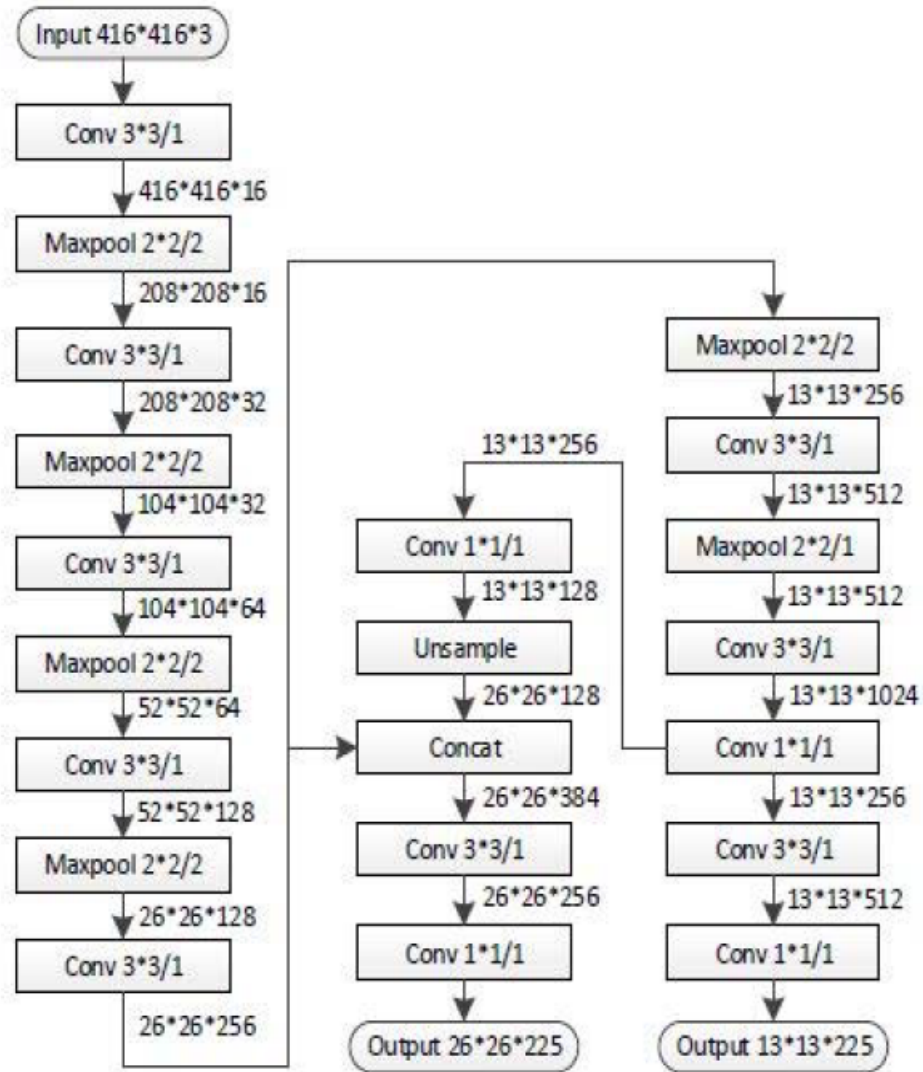


Figure 2.3: Network architecture of Tiny-YOLOv3.

2.1.3 Tinier-YOLO : Application of A real-time Object detection method in a constrained environment

2.1.3.1 Introduction:

Wei Fang Et al.[6] Used Tiny yolov3 as base and developed a network almost 4x smaller than tiny yolo v3, named it as Tinier-YOLO. This network being small still achieve 25 FPS real time object detection performance on NVIDIA TX1 platform with 65.7% mAP and 34% mAP on PASCAL VOC

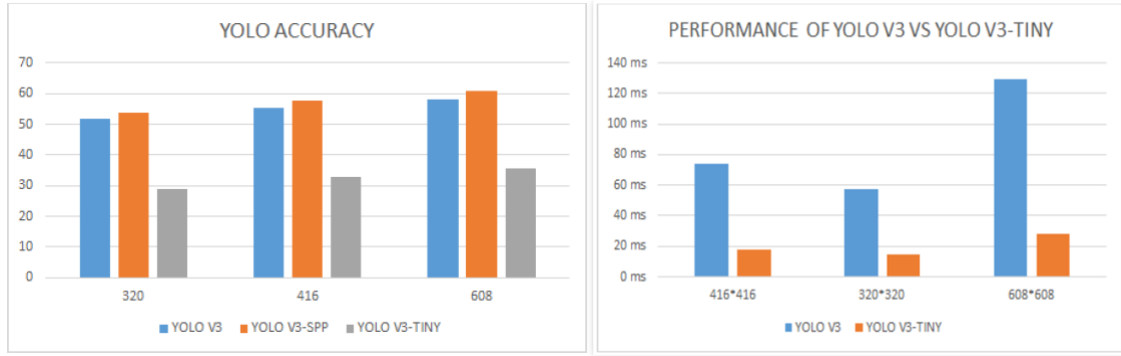


Figure 2.4: Performance of YOLOv3 vs Tiny-YOLOv3.

and COCO dataset.

2.1.3.2 Architecture:

Architecture of Tinier-YOLO retains front-five convolution layer called First part, Then middle part, which applies five-Fire modules which are densely connected. Fire module reduces amount of model-parameters and increases width-and-depth of layers. Denser connections with stronger feature extraction capability ensures maximum info flow, there by helps to improves model in increasing accuracy. Middle part has a pass through layer brings higher level features and merges with output of 5th fire module. This does not incorporate too many parameters but considerably improves accuracy of detection.

The authors also evaluated that removing batch normalization layer may not affect accuracy, but can reduce model size by 0.1mb, So the batch norm layer has been eliminated. Authors conducted experiments and compared results against Tiny-YOLOv3 using NVIDIA Titan X and Jetson TX1 devices.

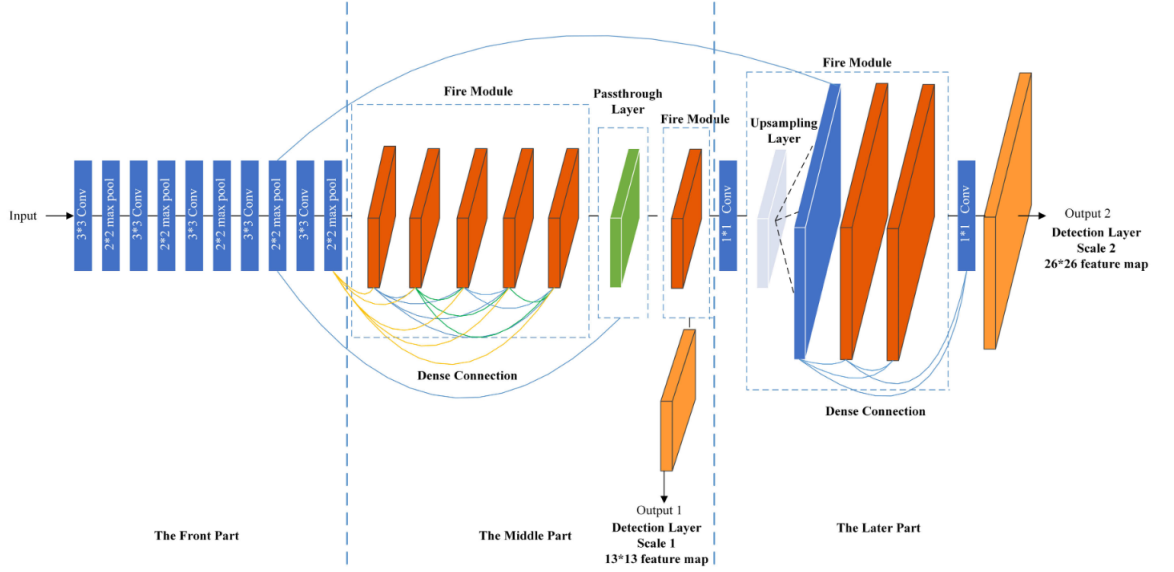


Figure 2.5: Network architecture of Tinier-YOLO.

	Tiny-YOLO-V3		Tinier-YOLO	
Fire Module		✓	✓	✓
Dense Connection			✓	✓
Passthrough Layer				✓
Model Size(MB)	34.9	6.9	8.3	8.9
BFLOP/s	5,474	2,350	2,552	2,563
mAP(%)	61.3	60.6	64.8	65.7
FPS	21.6	25.5	24.9	25.1

Figure 2.6: Comparison of experiments using Tiny-YOLOv3 and Tinier-YOLO.

2.1.4 YOLOv4

2.1.4.1 Introduction:

The next version of YOLO , YOLOv4 [7] was developed to fulfill the necessity of faster real time object detection along with improving the parallel computation by the authors.

One of the main objectives of YOLOv4 developers was that it could be used by anyone with minimum hardware requirements like a conventional GPU to make this version of YOLO more easy to access and to be customized by users to their needs.

As is with other YOLO versions, YOLOv4 is a model for object detection

with added on features that improve the performance over it's predecessors with prime concern being real time object detection.

The major difference between YOLOv3 and YOLOv4 is in the backbone and neck of the architectures.

YOLOv4 follows similar pattern in the skeleton of it's architecture, i.e. it consists of a backbone, neck and head.

It also utilizes 'bag of freebies' and 'bag of specials' which helps to increase the performance.

2.1.4.2 Architecture:

The components of YOLOv4 architecture is :

- **Backbone:** The backbone [8] of a model is important as it helps in feature extraction and is usually a pre-trained on imagenet classification. The backbone of YOLOv4 is CSPDarknet53 [9].

The authors experimented with various backbones like CSPResNext50, EfficientNet-B3 and CSPDarknet53 and with experimentation results and theoretical support they went with CSPDarknet53 as backbone for YOLOv4.

CSPDarknet53: It is based on CSPNet with DenseNet and Darknet-53.

CSPNet[10] : CSPNet was designed to help architectures get a good gradient without increasing the computation cost. To achieve this the feature map of the base layer is partitioned into two parts and then joined or merged them using a cross-stage hierarchy. This makes the gradient flow move through different network pathways by splitting it.

DenseNet[11] : DenseNet connects connects each layer to every other layer in a feed-forward fashion. It is called DensNet due to dense connectivity which is achieved through making direct connections from any layer to all the next layers.

Darknet-53[12] : Darknet-53 is called as it contains 53 convolutional layers. It was first introduced in YOLOv3 and performs better than it's predecessor Darknet-19 and others like ResNet-101 or ResNet-152

- **Neck:** For Neck YOLOv4 uses Path Aggregation Network[13] and a block before reaching neck is added which is Spatial Pyramid Pooling[14]. Neck is used for feature aggregation.

Spatial Pyramid pooling: The output from the backbone will be sent to

SPP. SPP was designed with the aim to generate a fixed-size output irrespective of the input size thus removes the constraint of needing a fixed size image input. It usually works by pooling the features and then generates fixed length output which are given as inputs to next layer.

Path Aggregation Network : Path Aggregation Network (PAN) is an advanced version of FPN. First the path through which information flows had to be shortened. Then authors proposed to add a bottom-up path augmentation along with the top-down FPN path. Also to handle and get back the broken information within all the feature levels adaptive feature pooling was introduced. It aggregated features from all feature levels.

- **Head:** YOLOv4 deploys the same YOLO head as YOLOv3[12] for detection with the anchor based detection steps, and three levels of detection granularity.

One of the new ways YOLOv4 was improved was by the introduction of BoF and BoS

- **Bag Of Freebies** Bag of freebies are so called that because they only change training cost. As conventional Object detectors are trained offline, this can be taken advantage of to create efficient training methods to increase the accuracy of object detectors. And since these methods don't create inference cost they are called as BoF. Data Augmentation is considered is one process that meets the definition of the same. In YOLOv4 the BoF is

Bag of Freebies (BoF) for backbone: CutMix and Mosaic data augmentation, DropBlock regularization, Class label smoothing.

Bag of Freebies (BoF) for detector: CIoU-loss, CmBN, DropBlock regularization, Mosaic data augmentation, Self-Adversarial Training, Eliminate grid sensitivity, Using multiple anchors for a single ground

- **Bag Of Specials** : Bag of Specials or BoS is another optimization method in YOLOv4. As opposed to BoF the BoS increases the inference cost by a small amount but at the same time can improve the accuracy of Object Detection as well. These modules are for improving certain attributes in a model, such as enlarging receptive field, introducing attention mechanism, or strengthening feature integration capability, etc., and post-processing is a method for screening model prediction results.

Bag of Specials (BoS) for backbone: Mish activation, Cross-stage partial

connections (CSP), Multiinput weighted residual connections (MiWRC)

Bag of Specials (BoS) for detector: Mish activation, SPP-block, SAM-block, PAN path-aggregation block, DIOU-NMS

After the YOLOv4 model was developed many implementations and sub versions were developed such as YOLOv4 - Tiny [15] proposed to implement YOLOv4 on mobile and embedded devices and YOLOv4- MT [16] uses mobile net and YOLOv4.

2.1.5 YOLOv5

2.1.5.1 Introduction:

The 5th generation of YOLO, YOLOv5, was developed by Glenn Jocher from **Ultralytics** using the **Pytorch** framework.

YOLO V5 [17][18] is a novel Convolution neural network (CNN) that detects objects in real-time with great accuracy.

This approach uses a single neural network to process the entire picture, then separates it into parts and predicts bounding boxes and probabilities for each component. These bounding boxes are weighted by the expected probability. The method “just looks once” at the image in the sense that it makes predictions after only one forward propagation run through the neural network.

It then delivers detected items after non-max suppression (which ensures that the object detection algorithm only identifies each object once)

2.1.5.2 Yolo V 5 Architecture:

Like any other YOLO Architecture V5 Consists of mainly three parts

- **Backbone:** Model Backbone is mostly used to extract key features from an input image. CSP(Cross Stage Partial Networks) are used as a backbone in YOLO v5 to extract rich useful characteristics from an input image o CSPNet has shown significant improvement in processing time with deeper networks
- **Neck:** The Model Neck is mostly used to create feature pyramids. Feature pyramids help models to generalized well on object scaling. It helps to

identify the same object with different sizes and scales. Feature pyramids are quite beneficial in assisting models to perform effectively on previously unseen data. There are other models that use different types of feature pyramid techniques like FPN, BiFPN, PANet, etc. In YOLO v5 PANet is used for as neck to get feature pyramids

- **Head:** The model Head is mostly responsible for the final detection step. It uses anchor boxes to construct final output vectors with class probabilities, objectness scores, and bounding boxes. In YOLO v5 model head is the same as the previous YOLO V3 and V4 versions.

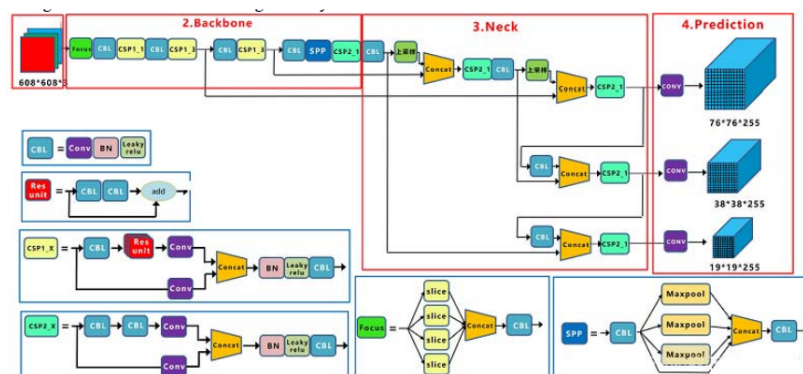


Figure 2.7: YoloV5 Architecture

2.1.5.3 Loss Functions used in YOLO5:

In the YOLO family, there is a compound loss is calculated based on objectness score, class probability score, and bounding box regression score. Ultralytics have used **Binary Cross-Entropy with Logits Loss function** from PyTorch for loss calculation of class probability and object score.

2.1.5.4 Binary Cross Entropy in YOLO5:

Binary crossentropy is a loss function that is used in binary classification tasks. These are tasks that answer a question with only two choices (yes or no, A or B, 0 or 1, left or right). Several independent such questions can be answered at the same time

$$\text{Loss} = -\frac{1}{\text{output size}} \sum_{i=1}^{\text{output size}} y_i \cdot \log \hat{y}_i + (1 - y_i) \cdot \log (1 - \hat{y}_i)$$

Figure 2.8: Binary Cross Entropy Formula

2.1.5.5 Logits in YOLO5:

A **Logit** function, also known as the log-odds function, is a function that represents probability values from 0 to 1, and negative infinity to infinity. The function is an inverse to the sigmoid function that limits values between 0 and 1 across the Y-axis, rather than the X-axis. Because the Logit function exists within the domain of 0 to 1, the function is most commonly used in understanding probabilities

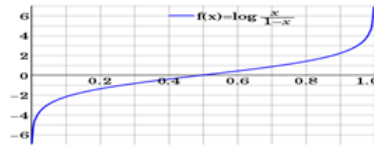


Figure 2.9: Logits Formula

2.1.5.6 Improved Loss Functions for Imbalanced Data

Even though most of the object detectors work well on common datasets, such as MS COCO [5], they usually suffer from two main problems: the imbalance between the number of background foreground data and infrequent observation of trained foreground object representations in the test set, i.e. the tail problem. To overcome this [19] proposed combined hard example mining structure which uses Balanced Focal Loss and Loss Rank Mining

2.1.5.7 Balanced Focal Loss

The popular cross entropy loss function is shown in (1), whereas original Focal loss function, proposed by Lin et. al.

$$CE(p) = -\log(p) \quad (1)$$

$$FL(p) = -\alpha(1-p)^\gamma \log(p) \quad (2)$$

Figure 2.10: Improved Loss Formula

2.1.5.8 Loss Rank Mining

In its original paper, this method is used with YOLOv2, which has one feature map for object detection. Since YOLOv5 uses three feature maps for small, medium and large objects, the original method is also modified in such a way that it works with all three feature maps. In the original LRM structure, first K detection results with the highest amount of loss are selected. In our method, first B (rank factor) detections are selected for each feature map. The comparison between the proposed combined objectness loss structure and the original YOLOv5 loss is illustrated in Figure 1 and the method can be summarized as follows for each feature map:

Through each mini-batch, 1. Balanced Focal Loss is applied to cells for obtaining loss values of the detections. 2. By flattening the three-dimensional cell structure, loss values of each image sample are concatenated into different vectors separately. 3. Loss values of each image are sorted by a value. 4. From the sorted loss vectors, the top B proportion of the number of cells for each image sample is selected. 5. The mean of each selected loss is taken individually. 6. These averages are summed to form the objectness loss

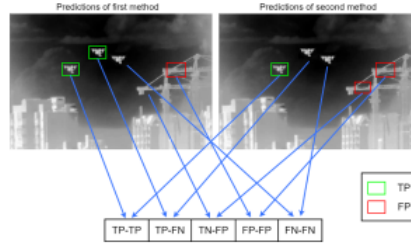


Figure 2.11: Improved Loss Function

2.1.5.9 Experiments

The paper also conducts an experiment using YOLOv5 as the baseline detector the and documented the results as shown below

Method	Prec.	Rec.	mAP _{0.5}	mAP _{0.5:0.95}
Default	98.0	85.4	90.4	53.3
Focal Loss	93.6	84.3	90.4	53.3
Bal. Focal Loss	98.2	89.6	92.6	55.9
LRM	98.0	90.0	93.2	57.3
Combined	98.3	91.0	93.5	56.1

Figure 2.12: Performance Evaluation of Baseline vs Proposed

2.1.5.10 Conclusion

Two hard example mining methods are modified and adapted on a state-of-the-art object detector, YOLOv5. The experiments clearly indicated that although the original focal loss degrades the precision of YOLOv5, the proposed Balanced Focal Loss corrects such inaccuracies and improves the overall performance.

2.1.5.11 Advantages and Disadvantages of YOLO V5:

- * YOLO V5 is about 88 times smaller than YOLOv4 (27 MB vs 244 MB)
- * YOLO V5 is about 180 times faster than YOLOv4 (140 FPS vs 50 FPS)
- * YOLO V5 is roughly as accurate as YOLOv4 on the same task (0.895 mAP vs 0.892 mAP)
- * The main drawback for YOLO V5 is that there is no official paper was released like other YOLO versions. Also, YOLO v5 is still under development and we receive frequent updates from Ultralytics, developers may update some settings in the future

2.1.6 YOLO FIR and YOLO FIRI

2.1.6.1 Introduction:

To solve object detection issues in infrared images, such as a low recognition rate and a high false alarm rate caused by long distances, weak energy, and low resolution, [20] proposed a region-free object detector named YOLO-FIR for infrared (IR) images with YOLOv5 core by compressing channels, optimizing parameters, etc. An improved infrared image object detection network, YOLO-FIRI, is further developed

2.1.6.2 Network Architecture:

YOLO FIRI network architecture mainly includes three parts:

Backbone network,
Neck,
Detection head.

The input single-channel infrared image is extracted through the backbone network with the extended CSP module. Multiscale features are further fused in the neck. Finally, the four-scale feature maps obtained are used to achieve multiscale object detection in the detection head

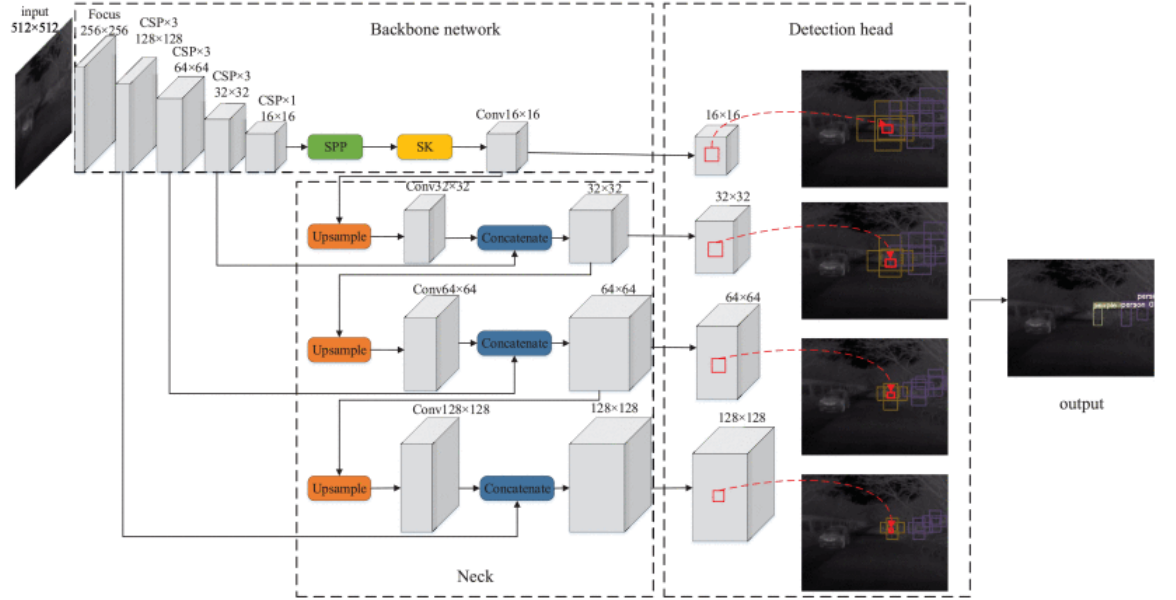


Figure 2.13: YOLO FIRI Architecture

In Figure 1, the input infrared image changes from $512 \times 512 \times 1$ (infrared images have a single channel) to $256 \times 256 \times 4$ after the focus operation. Then, in the backbone network, the extended CSP module is used to extract rich information from shallow and deep feature maps after the focus operation, and the attention mechanism introduced in the CSP module guides the assignment of different weights to realize and notice the extraction of weak and

small features. In addition, the SPP (Spatial Pyramid Pooling) layer can concatenate the results obtained in the channel dimension through four pooling windows to solve the alignment problem of anchors and feature maps. We use the SK attention module to enhance the extracted features. Second, in the neck network, PANet (Path Aggregation Network) is used to generate feature pyramids, and the top-down and bottom-up fusion structures are both used to effectively fuse the multiscale features extracted from the backbone network and enhance the detection of objects with different scales. Finally, in the detection head, the four sets of output feature maps are detected, and the anchor boxes are applied on the output feature maps to generate the final output vectors with a class probability score, a confidence score, and a bounding box. Then, according to the NMS postprocessing, the results detected by the four detection layers are screened to obtain the final detection results. The additional set of feature maps can solve the problem of missed and false detections caused by long-distance shooting. The proposed network models demonstrate a qualitative improvement compared with the latest infrared image object detection in terms of detection accuracy, reasoning speed, and network parameters

2.1.6.3 Comparison of the Detection Performance on KAIST infrared pedestrian dataset

Quantitative Comparison of YOLO-FIRI and the Other State-of-the-Art Object Detectors. The Results are Reported in Terms of AP, AR, mAP50, and F1 Percentage to Evaluate the Accuracy, and Detection Times, Parameters, Weight Size, and FLOPs to Assess the Speed. All the Experiments Use the KAIST Test Set. The Speed and Accuracy of the Proposed Models are Qualitatively Improved Compared With the State-of-the-Art Infrared Images Object Detection Algorithms

Method	AP/%	AR/%	mAP/%	F1/%	mAP50:75/%	Time/ms	Params/M	Weight/MB	FLOPs/B
YOLOv3	73.5	76.9	79.6	74.8	57.0	25	61.5	246.4	155.1
YOLOv4	76.9	75.8	81.0	76.3	58.9	37	63.9	256.3	128.4
YOLO-ACN	76.2	87.9	82.3	81.6	59.3	20	47.4	177.6	111.8
YOLO-FIR	92.1	88.1	93.1	90.0	82.4	12	7.1	16.2	16.4
YOLO-FIRI	96.0	96.2	98.3	96.1	95.6	14	7.2	15.0	20.4

Figure 2.14: YOLO FIRI Performance

2.1.7 YOLOv6

2.1.7.1 Introduction:

Yolo V6 [21] is a single stage object detection framework which can be used for Industrial applications. That means given any image, region detection and Object classification happens in a single network which is single stage object detection. The advantage of single stage object detection over two stage object detection is that we get better performance but may have slightly lesser accuracy than two stage.

As in any YOLO model, YOLO V6 also uses 3 components which is Backbone, Neck and Head. Backbone extracts the features from different resolution of image, Neck performs the feature aggregation and Head performs the actual classification or regression.

Backbone in YOLO architecture is very vital as it provides inputs for Neck and Head and quality of Backbone inputs has a direct impact on the predictions by the Head.

2.1.7.2 Architecture of YOLO6 Model:

2.1.7.2.1 YOLO6 Backbone Architecture: YOLOv6 models use reparameterized backbones. In reparameterization, the network structure changes during training and inference. This takes care of lower performance of ResNet at the inference phase or lower accuracy of VGG.

In YOLO v6 Nano, tiny and small architectures use reparametrized VGG block with skip connections during training phase and uses 3X3 convolutional blocks during inference. For medium and large models, it uses reparameterized CSP backbone.

Performance and Accuracy of the models improve as the size of the model increases.

2.1.7.2.2 YOLO6 Neck Architecture: YOLOv6 models also use Path Aggregation Network like earlier YOLO model to aggregate the feature maps. Since it aggregates various re-parameterized blocks it is called RepPAN
YOLO6 Head Architecture: There is a significant change in this component of YOLOv6 models where classification and regression do not share the same parameters and branch out separately in backbone itself. This is the main reason for higher performance and higher accuracy.

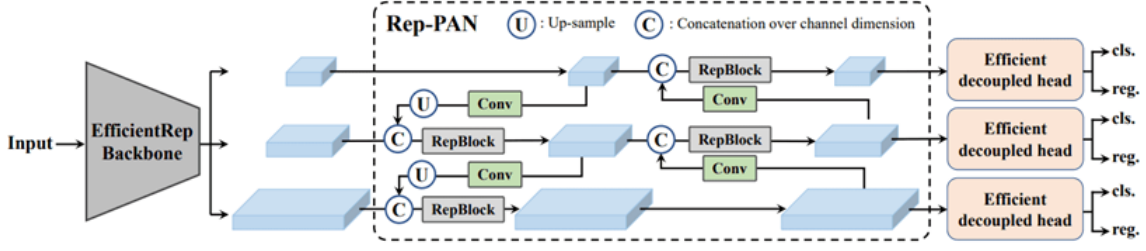


Figure 2.15: YoloV6 Architecture

2.1.7.3 Loss Functions used in YOLO6 Model:

Having got 2 different usages of this model, we have 2 loss functions in YOLO 6 models.

- VFL (Varifocal Loss) [22] for measuring the loss when classification part is used
- DFL (Distribution Focal Loss) along with SIoU or GIoU as box regression loss when we use regression

Varifocal Loss is a loss function for training a dense object detector to predict the IACS. Unlike the focal loss that deals with positives and negatives equally, Varifocal Loss treats them asymmetrically.

$$\text{VFL}(p, q) = \begin{cases} -q(q \log(p) + (1 - q) \log(1 - p)) & q > 0 \\ -\alpha p^\gamma \log(1 - p) & q = 0, \end{cases}$$

Figure 2.16: Formula for Varifocal Loss

where p is the predicted IoU Aware Classification Score and q is the target IoU score.

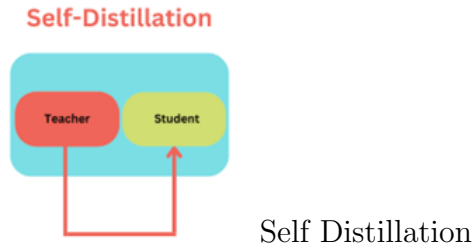
YOLOv6 Medium and Large models use DFL for box regression loss. DFL treats the continuous distribution of box locations as a discrete probability distribution. It is especially helpful in detection when the boundaries of the ground truth are blurred.

YOLO6 Applications for solving Industrial Problems YOLOv6 is extensively used for solving industrial applications, and there are many improvements to achieve the same

Training Time and resources are spent more to provide more training to the models. These models are trained for atleast 30

YOLOv6 uses knowledge distillation [23] to further improve the accuracy of the models. This significantly reduces the cost of building a complex model and use complex model only as a Teacher and fine tune the model. Knowledge Distillation consists of two neural networks: Teacher and Student

models. Teacher Model: A larger cumbersome model can be an ensemble of separately trained models or a single very large model trained with a very strong regularizer such as dropout. The cumbersome model or the Teacher model is trained first.



2.1.8 YOLOv7

Yolo V7 [24] introduced in July 22 has improved architecture which is more accurate and also faster. The paper states that Yolo V7 can predict video inputs ranging from 5 FPS to 160 FPS ⁸ and has the highest accuracy of 56.8 percentage average precision ⁹. Yolo V7 outperforms both transformer-based object detectors and convolution based object detectors. Some of the object detectors that Yolo V7 outperform YOLOR, YOLOX, Scaled-YOLOv4, YOLOv5, DETR, Deformable DETR, DINO-5scale-R50 and ViT-Adapter-B in speed and accuracy. Yolo V7 has 75 percent fewer parameters and 36 percent lesser computational time than Yolo V4 with 1.5x times higher average precision. The author uses gradient prediction to generate coarse define hierarchical labels. The author also uses extended efficient layer aggregation networks. The author also performed model scaling for concatenation based models and identifying connections in one convolution layer. The author used Microsoft Coco data set to train the Yolo V7 from scratch without using any other data sets or pre trained weights. The model is able to perform with these pre trained weights only using Microsoft Coco data set. During the research it was figured out that the average precision was higher when the IOU threshold ¹⁰ was increased.

Figure 2.17[25] shows the overall network architecture of yolov7. It can be

⁸FPS :refer section 3.0.6.1

⁹mAP :refer section 3.0.6.3

¹⁰IOU :refer section ??

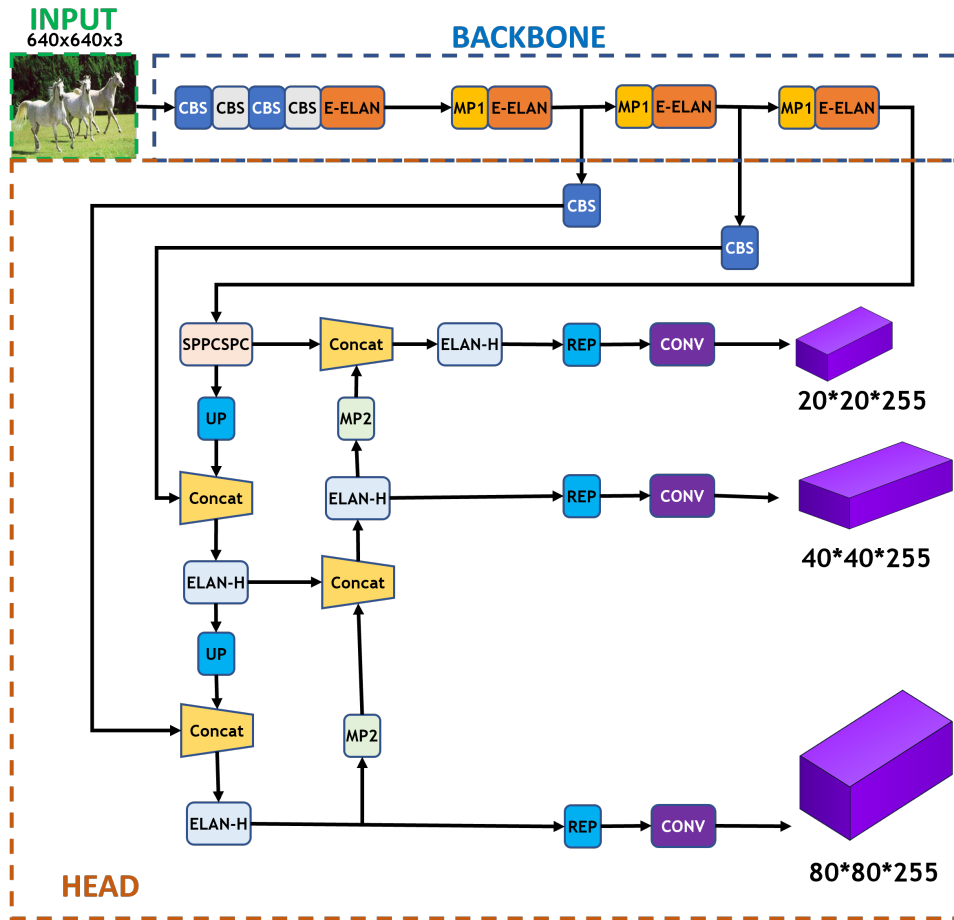


Figure 2.17: YoloV7 Architecture

seen from the figure that the yolov7 network consists of three parts: input, backbone and head . its different from yolov5 as the neck layer and the head layer are collectively called the head layer however actual functions are the same . The functions of each part are the same as yolov5, such as backbone for feature extraction and head for prediction.

The backbone of YOLOv7 consist stacking of multiple CBS (Conv¹¹ + Batch-Norm¹² + SiLU¹³) modules and E-ELAN (Extended Efficient Layer Aggregation Network)

YOLOv7 paper introduces the following major changes.

¹¹Conv: refer section 3.0.1.1

¹²Batchnorm: refer section 3.0.1.4

¹³Activation: refer section 3.0.2

2.1.8.0.1 Extended Efficient Layer Aggregation Network (E-ELAN)

The efficiency of a Yolo networks convolutional layers in the backbone is essential to efficient inference speed. Earlier versions used cross state partial networks for maximum efficiency. The authors researched and created an efficient backbone that led to efficient model training which eventually leads to more better model accuracy. the following factors were considered in re-design of this backbone architecture.

- 1) memory accessed and its cost.
- 2) Storing of weights in layers and model parameters
- 3) Channel ratio, that is how many number of channels are there in input layer and how many after passing through a certain convolutional block versus output channel.
- 4) Computational blocks and how efficiently the gradients are calculated when they are back propagated from the output layer.

Initially the authors used other backbone like resnet and efficient net, eventually started using concatenation-based models like dense net. The main purpose of dense net is overcoming the problem of vanishing gradient, where when the model more deeper and deeper, during back propagation keeping the gradient significant becomes challenging. Output feature maps were aggregated and passed to layers before. To overcome redundancy cause by every layer feedback , the authors proposed one shot aggregation called Efficient layer aggregation network.

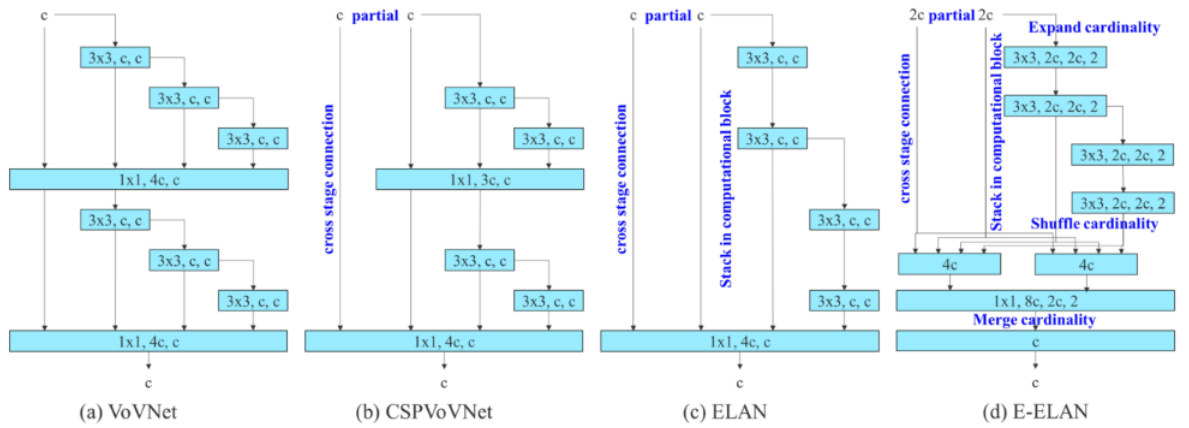


Figure 2.18: Extended Efficient Layer Aggregation Network

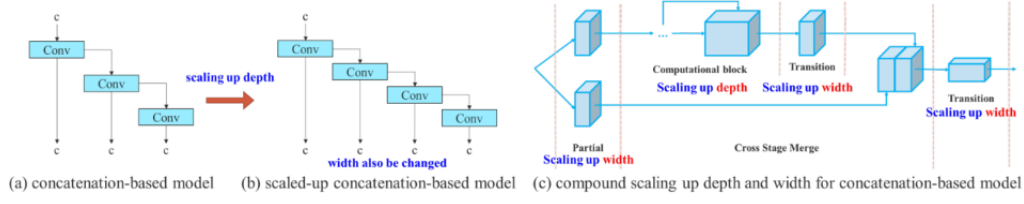


Figure 2.19: Model scaling for concatenation-based models

2.1.8.0.2 Scaling for Concatenation based Models All concatenation-based model changes the input dimensions, in yolov7 architecture using various levels of concatenation coupled with convolution the length and width of image is reduced, and depth is increased. this enables the model to identify small and large object depending on the resolution, depth length and width.

2.1.8.0.3 Rep convolution Re-parameterization [26] can be done at model or module level using ensemble technique which is aggregation of different module weights. Yolo V7 uses ensemble of three different convolution module blocks during training and aggregate to single model weight. As shown in the figure authors experimented on different module combinations to come to the most accurate training weights. During deployment, the single aggregated weight module makes Yolo V7 fast in detection of images.

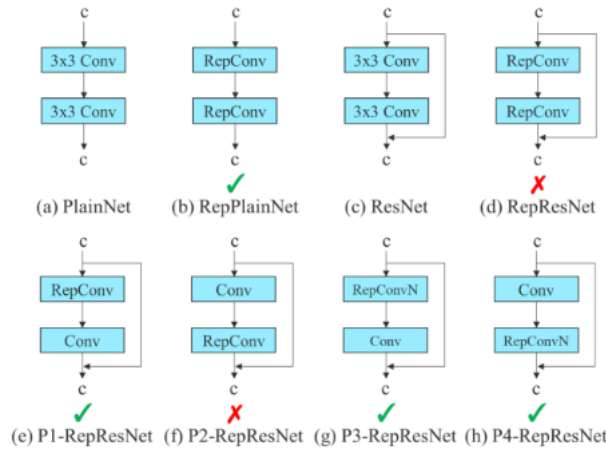


Figure 2.20: Planned re-parameterized model

2.1.8.0.4 Lead and Auxiliary Head In conventional models the final output of the model called hard labels is compared with the ground truth to calculate loss and derive the model weights. The authors proposed a model with lead head and auxiliary head in which the output of the lead head and ground truth was combined to form an intermediate soft label. this soft label was used to calculate the loss with respect to output of auxiliary head and fed as input to lead head thereby enabling the model to converge faster and improve performance.

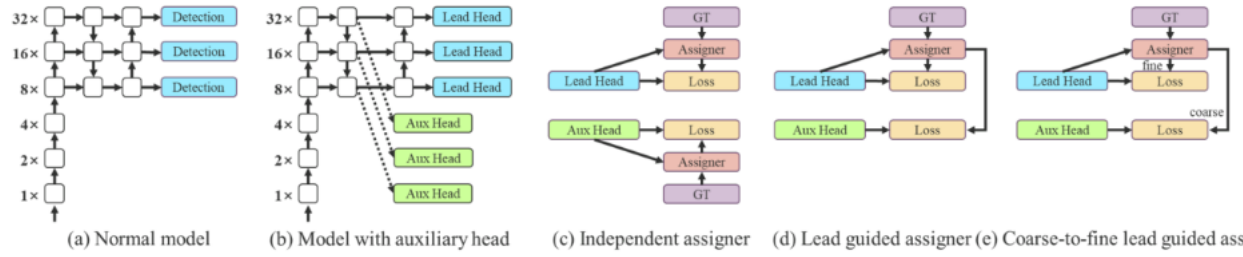


Figure 2.21: Coarse for auxiliary and fine for lead head label assigner.

2.2 Yolo use cases

2.2.1 Object Detection

2.2.1.1 Vision Based Intelligent Shelf-Management System

The current trend of people flocking to the supermarkets ignoring the local stores has created a potential opportunity to use Machine Learning and Artificial Intelligence capabilities in supermarkets and high end stores to identify the empty shelves, percentage of availability of items on the on shelf which may in turn be used to predict future sales. This approach can also be used to reduce the existing time-consuming methods of using manual labor to detect the empty shelves. Empty-shelves in stores in turn affect customer satisfaction and store profitability. The author of the paper [27] introduces a cost effective and low time-consuming method to the owners of the stores and super markets by curtailing the problem of identifying the pattern of products running out of stock or the products removed from shelves. A camera placed in the shelf captures real time images at specific intervals. The proposed system detects the image which is similar to the set threshold factor and will alert the owners/staff that the shelf is running out of stock. Predictions for future supply are done using Time Series Forecasting based on existing data. The paper also addresses the challenges faced by owners due to lack of knowledge of demand and supply. Countries like Sri Lanka often faces these kind of challenges. The author states that the main aim of the research is to implement a Vision Based Intelligent -Shelf Management System dedicated to supermarket employees and owners. The author proposes to use machine learning as a better alternative to the existing manual stock keeping methods which can help businesses make informed decisions.

2.2.1.2 An attention mechanism-improved yolov7 object detection algorithm for hemp duck count estimation

Jiang et al.[25] proposed an improved attention mechanism YOLOv7 algorithm, CBAM-YOLOv7, adding three CBAM modules to the backbone network of YOLOv7 to improve the network's ability to extract features and label small object like ducks. Authors were able to develop a smart and reliable automated duck counting using Yolov7 architecture and custom dataset containing 1500 labels for the whole body frame and head frame for duck target detection. Data was preprocessed with Mosaic Data Augmentation. Mosaic data augmentation. Firstly, a batch of image data was randomly extracted from the dataset of mallard ducks. Then, four images were ran-

High Level Architecture

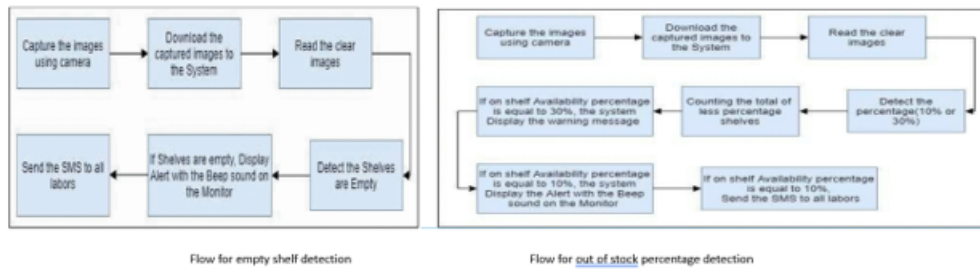
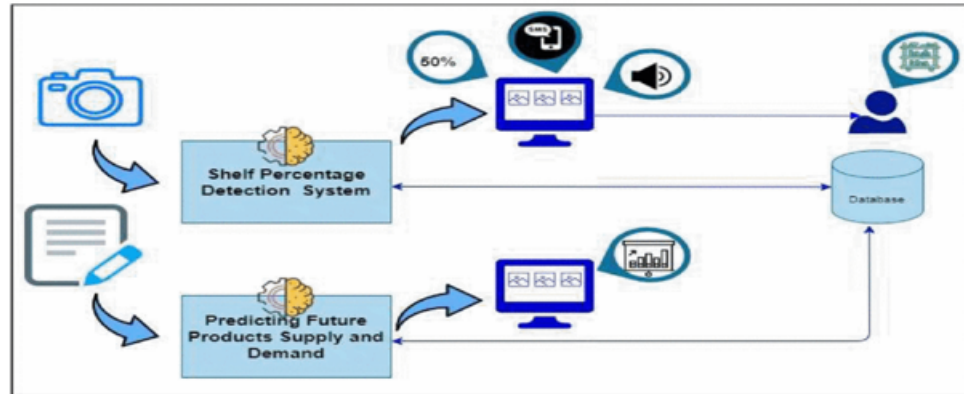


Figure 2.22: Vision Based Shelf Life System.

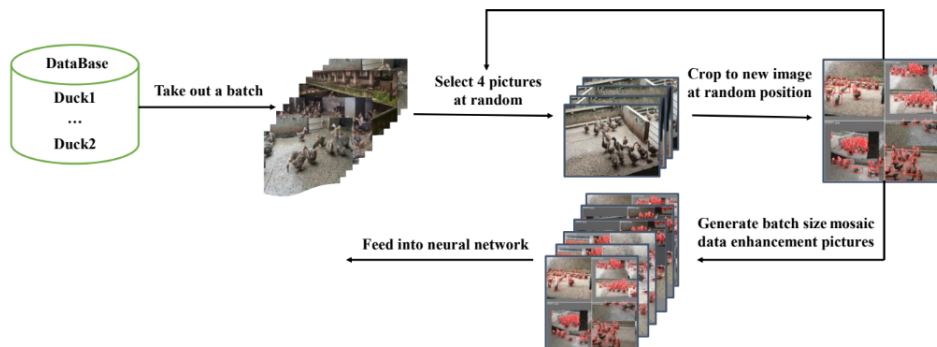


Figure 2.23: Mosaic Data Augmentation

domly selected, randomly scaled, randomly distributed, and spliced into new images, and the above operations were repeated for batch size times. Fi-

nally, the Mosaic data augmentation data were fed into the neural network for training.

2.2.1.3 A deep learning approach for detection and prediction of kidney diseases

Bayram Et al.[28] used YOLOv7 and YOLOv7 Tiny Architectural Design. in their study to improve the detection of cysts and stones in kidney. The authors used custom datasets of CT images in three classes, namely 72 renal cysts, 394 renal stones and 192 healthy kidneys were used in the performance analysis part of the YOLO architectural designs. As a result, the architectural design of YOLOv7 outperformed the architectural design of YOLOv7 Tiny. The YOLOv7 architecture design achieved an mAP50 of 0.85, an accuracy of 0.882, a sensitivity of 0.829, and an F1 score of 0.854.

2.2.1.4 Multi-Scale Detector for Accurate Vehicle Detection in Traffic Surveillance Data.

Kwang-Ju Et al [29] YOLOv3 based network architecture used with integration of Spatial Pyramid Pooling to compensate and enhance the detection accuracy of vehicle detection task which have large scale deviation or occluded by other objects. Authors Kwang-Ju Et al. [29] have tried to come up with improved method to analyze, understand and take necessary action to prevent various traffic problems and improve traffic flow efficiency. Authors propose multi-scale detector for efficient and accurate vehicle detection model using YOLOv3's Darknet-53 backbone network as it is and modifying next layers. The feature pyramid network has been modified with additional five - pooling layers in the Spatial Pyramid Pooling (SPP) blocks. There are additional prediction layers which cover objects of various scale. With customized scale authors design 15 anchor boxes ranging from 5 to 400 pixels. Early stage feature maps detect smaller object with smaller receptive area and later ones cover larger feature maps. Up-sampling is applied to effectively combine feature maps of earlier stages into later stage feature maps. After concatenation with SPP output a batch normalization is applied before feeding them to convolution layer. A Soft non-max suppression technique used to consider all detection as function of IOU of all objects which have maximum score. Soft-NMS reduced confidence scores of predicted BBoxes instead of discarding them completely and hence gives better average precision over conventional-NMS. Authors do qualitative evaluation

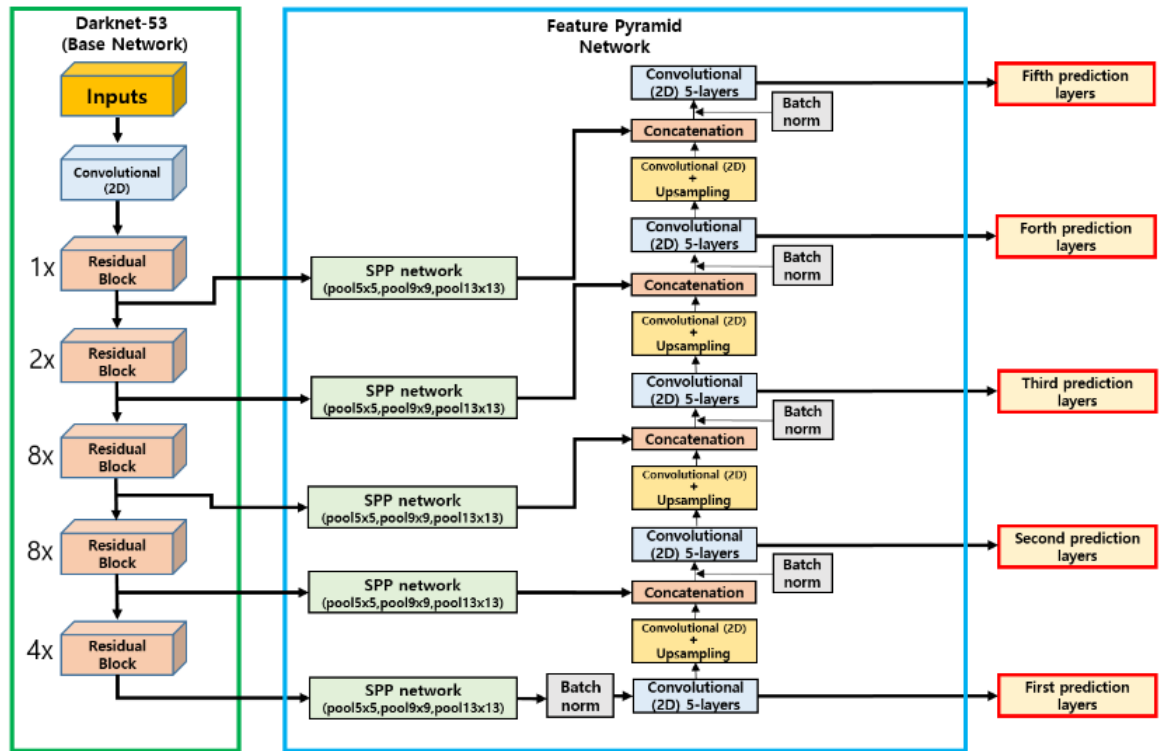


Figure 2.24: Multiscale YOLOv3 Object detection network architecture

of approach no UA-DETRAC benchmark test-sets. It is found that approach performs well even when heavy and partial occlusions are found and also when vehicles are far away from camera with a FPS of 9-10 with state of the art accuracy in YOLOv3.

2.2.1.5 Multi-Agent Deep Reinforcement Learning for Multi-Object Tracker.

[30] Multi-object tracking is key research topic in modern mobility solutions, mass surveillance, human action recognition and tracking etc. Authors Mingxin Jiang et al. propose a novel approach based on multi-agent deep re-enforcement learning (MADRL) for Mult-object tracking (MOT) tasks. The method uses deep leaning approach to detect objects first in each frame. These each detection form an independent agent in the current frame and treats other agents as part of environment. Here the job of detectors plays vital role in detecting object faster and accurately. Hence Yolov3 provides better detection.

2.2.1.6 Embedded System Real-Time Vehicle Detection based on Improved YOLO Network.

Majority of deep learning object detection models having higher accuracy tend to bulky, struggle to fit on to a embedded device which normally gets installed into automobiles, medical equipment deployed onsite. Authors Shaobin Chen et al. suggest newer, better network architecture based on YOLOv3-Tiny called YOLOv3-live [31]. To optimize network, feature extraction is performed with two convolution kernel of different scales. Compared to receptive field of 3x3, a 5x5 kernel has higher receptive field, which increases network parameters but also increases information flow. However 5x5 kernel are replaced by two consecutive 3x3 layers. This stricture is called as live module. Authors test this model on Kitti datasets and found that there improvement of mAP compared YOLOv3-tiny by 1.71%. The quantized version with bit of (15% mAP) compromise on accuracy performs x4 time faster than unquantized version.

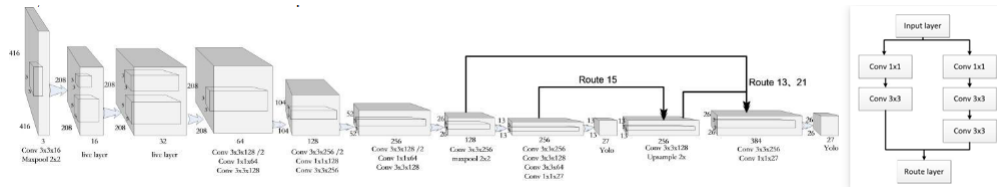


Figure 2.25: Yolov3-Live network architecture

2.2.1.7 A Deep Learning (YOLOv3) Approach for Street Pothole Detection.

Potholes [32] are structural anomalies that can cause trivial to serious damage or even could be fatal to traffic participants. In this paper authors Ping Ping et al. have done study on Pothole detection using deep learning approach. They have used four object detection models (YOLOv3, SSD, HOG+SVM and Faster-RCNN) as pothole detectors. Compared to machine learning and classical computer vision based approaches, Deep learning algorithms works on entire image making them to learn even the neighbouring context as well. Hence YOLOv3 generalizes well to detect potholes well compared to other approaches.

	precision	recall	F1-score	Avg IOU
YOLO v3-tiny	88%	81%	84%	73.12%
YOLO v3-live	91%	82%	86%	74.64%

	front	back	side	warning	mAP
YOLO v3-tiny	84.61%	81.35%	88.12%	90.24%	86.08%
YOLO v3-live	86.38%	82.70%	90.36%	91.72%	87.79%

	mAP	FPS
YOLO v3-live (unquantified)	84.15%	6
YOLO v3-live (quantified)	69.79%	28

Figure 2.26: Yolov3-Live Performance comparison with YOLOv3-Tiny

Size	YOLO V3	SSD	HOG	Faster R-CNN
200 Images	3 hours	4 hours	2 hours	2 hours
650 Images	4 hours	5 hours	3.5 hours	2.5 hours
850 Images	4 hours	5.5 hours	4 hours	3 hours
1000 Images	4 hours	6 hours	-	3.5 hours
1100 Images	4.5 hours	-	-	4 hours
1500 Images	5 hours	8.5 hours	-	6 hours

Size	YOLO V3	SSD	HOG	Faster R-CNN
200 Images	53%	47%	24%	72%
650 Images	67%	59%	25%	71%
850 Images	65%	55%	27%	67%
1000 Images	69%	59%	-	69%
1100 Images	73%	-	-	60%
1500 Images	82%	80%	-	74%

Figure 2.27: Pothole detection using Deep learning

2.2.2 Pose Estimation

2.2.2.1 YOLO-6D+: Single Shot 6D Pose Estimation Using Privileged Silhouette Information

With the increased research and advancements in technology, and applicability of Augmented reality and Virtual reality domains have gained traction in recent years. Companies like meta have invested billions in AR/VR technologies which could transform the way humans have seen the world around them. AR/VR have capability to provide a new experience to human brain which is currently constrained by many fundamental science rules like gravity, speed of light etc. Jiaa Kang et al. [33] have come up with a paper in which they use 6D object pose estimation technique using RGB images. A new network architecture named YOLO-6D+ was introduced which learns 6D pose from these RGB images and project a 6D pose of objects of interest. A custom called silhouette prediction branch was used to learn silhouette feature information. Authors have introduced a custom new loss function called edge restrain loss which specializes in learning 3D shapes of target objects. A two stage approach which predicts key points in 2D and then estimates 3D world positions with the help of PnP algorithm. Authors claim to achieved state-of-the-art performance on LINEMOD dataset based on YOLO-pose estimator.

2.2.3 Motion detection

2.2.3.1 A sport athlete object tracking based on deep sort and yolo v4 in case of camera movement

Zhang et al [34] performed object tracking based on Deep sort and Yolo V4. This paper takes NBA and World Cup-related scenes as an identification object and aims to set up a tracking system for all players in the game, complete the real-time tracking of each athlete to obtain relevant track information. This information can help coaches and athletes improve their performance. Unlike most filtering algorithms in the past, this paper used the more advanced deep learning technology, YoloV4 and Sort's enhanced version Deep Sort. INRIA Person Dataset was used to train Yolo and max IOU of 0.97673 was obtained. Market1501 and MARS datasets were used to train DeepSort and mAP of 0.682 was obtained.

2.2.3.2 Visual object detection and tracking using yolo and sort

Bathija et al. [35] trained custom image data set consisting of 800 images for 6 specific classes with YOLO and this model tracked videos by using SORT algorithm for detecting a vehicle or pedestrian [36] in a running video is helpful for traffic analysis.

2.2.4 Human Activity Recognition

2.2.4.1 Yolo based human action recognition and localization

Shubham Shinde et al.[37] presented an approach for real-time detection, localization and detection of interesting human actions from still images obtained through a continuous stream of video data that can be captured by a surveillance camera. The model takes input frames after a period of time and is given an action label based on a single frame. Combining results over time predicts action labels for the video stream. The authors showed that YOLO is an effective and comparatively fast method for detection and localization in the Liris Human Activities dataset. The model was trained up to 40000 iterations and average loss was found to be 0.32 for the batch size of 32 with 8 subdivisions.

2.2.4.2 New action recognition frame- work for video highlights summarization in sporting events

Cheng Yan et al.[38] proposed an action detection framework for summarizing video highlights in sports events using Yolo. The study aims to develop an annotation system that uses a highly accurate framework to automatically trim the sports video stream by using a three-stage prediction algorithm based on YOLO-v3 and Open Pose [39]. It has been found that the proposed methodology can perform accurate clipping of sport activity highlights using a modest amount of sport video training data. Custom videos of table tennis matches were used to train the model and scored against three criteria, namely average precision, recall, and combined metric precision to achieve approximately 0.5858, 0.4958, and 0.5267, respectively.

2.2.4.3 Skeleton based temporal action detection with yolo

Jun Wu et al [40]carried out human activity recognition with Skeleton Based Temporal Action Detection using YOLO. The authors transformed

temporal action detection problem into the object detection problem. Their method enables real-time detection and end-to-end training. It consists of two stages. First, authors proposed an idea to represent action sequences as images while preserving the original temporal dynamics and spatial structure information. Second, based on such a description, they designed a one-dimensional YOLO network to detect human actions. The authors collected datasets for 7 actions (sit to stand, stand to sit, lie down, stand up, eat, read, and drink). In order to improve the ability to generalize, 3 samples persons in 2 scenes were selected that take into account the environment and people factor. The final mAP .8188at IOU=0.5 was obtained.

2.2.4.4 Robot recognizing humans intention and interacting with humans based on a multi-task model combining st-gcn-lstm model and yolo model

Chunfang Liu et al. [41] developed a multi-task model for human intent detection that consists of two subtasks: human action detection and hand-held object identification. For the first subtask, an effective ST-GCN-LSTM model is proposed by fusing the Spatial Temporal Graph Convolutional Networks and Long Short Term Memory Networks and for the second subtask, the YOLO v3 model for handheld object identification is adopted. The authors then build a framework for the robot's interaction with the human. Finally, these proposed models and the interacting framework are verified on the SBU Kinect Interaction Dataset, which contains 230 sequences of 8 classes (6614 frames) with subject-independent 5-way cross-validation for each subject and a custom dataset created by authors based on a Kinect V2 camera was created.

2.2.4.5 target detection network for dangerous driving gestures

Yefan Zhou et al. [42] developed a model for detecting dangerous driving gestures . To achieve this the Yolov4's target detection network is improved upon and the model DDGNet-YOLO was built. DDGNet stands for Dangerous Driving Gestures Network - YOLO. The DDGNet - YOLO uses residual learning method to handle gradient loss and it also is a improvement to network speed. For activation function , Mish activation is used. The data set used is called CVRR-HANDS 3D

2.2.4.6 Abnormal behavior detection in real-time for advanced driver assistance system (adas) using yolo

May Thu Soe et al. [43] has proposed a model for detecting drowsiness and abnormal behaviour while driving for Advanced Driver Assistance System called ADAS. By leveraging YOLO's incredible object detection they have come up with a method to detect abnormal behaviour while driving. They have a custom CNN concatenated with YOLOv2's detection. The custom CNN is used for feature extraction and YOLOv2 is used for detection.

2.2.4.7 Driver drowsiness detection and alert system using yolo

Yogesh R et al. [44] has used YOLOv3 to detect real time drowsy state of a driver. Facial land mark and key points [45] detection has become popular research area and has enormous real-time applications. First the driver's face is recorded and facial landmarks are detected. They include eyes closing, yawning and eating. Then the image captured goes through further processing where finally it is detected if the driver is drowsy or not. The YOLOv3 is used to extract features along with a tool called Dlib.

2.2.4.8 Human fall detection using yolo: A real-time and ai-on-the-edge perspective

Sergio A. Velastin et al. [46] Human Fall Detection YOLO: In this paper from Dr Velastin's group uses YOLO v4 for elderly fall detection. With increasing aging population globally, this study is the need of the hour. This research was done to enhance the safety in elderly population living independently in a practical manner. In this study, images are annotated and converted to weights using Tiny YOLOv4. Data was obtained from a publicly available dataset consisting of 70 sequences of daily activities – 40 activities of daily living and 30 activities that involve a fall. The converted weight is transferred to Raspberry-Pi a monitoring device. All versions of Yolo architectures were deployed and compared. Tiny – Yolo V4 gave best results when compared to other Yolo models. It gave a precision of 95 percentage, Recall of 96 percentage and F1-score of 95 percentage

2.2.4.9 Safety helmet detection method based on YOLO v4

Dr Miao et al . [47] In this paper from Dr Miao's group, YOLO v4 is used for testing if construction workers are wearing safety helmet. Wearing safety helmet constantly while working in construction sites, is crucial in avoiding workplace hazards. In this study, they use a self made dataset of CCTV footage obtained from the construction site. They then use K- means clustering to obtain appropriate frame dimensions and targeted edge information. They use YOLO v4 and are able to detect the presence of safety helmets with an accuracy of 92.9 percent with detection speed of 15 frames/second

2.2.4.10 A single shot deep pose estimation and analysis

Dr Nahavandi et al . [48] In this paper by Dr Nahavandi, YOLOv3 is used for human deep posture analysis. The authors have proposed SSDPose which is an end to end trainable model capable of detecting and estimating human posture and form from a single image. This is to improve ergonomics in manufacturing industries and factories. SSDPose uses person detection and pose estimation in one regression model. Here authors use DarkNet53 for feature extraction. Convolutional pose machine architecture

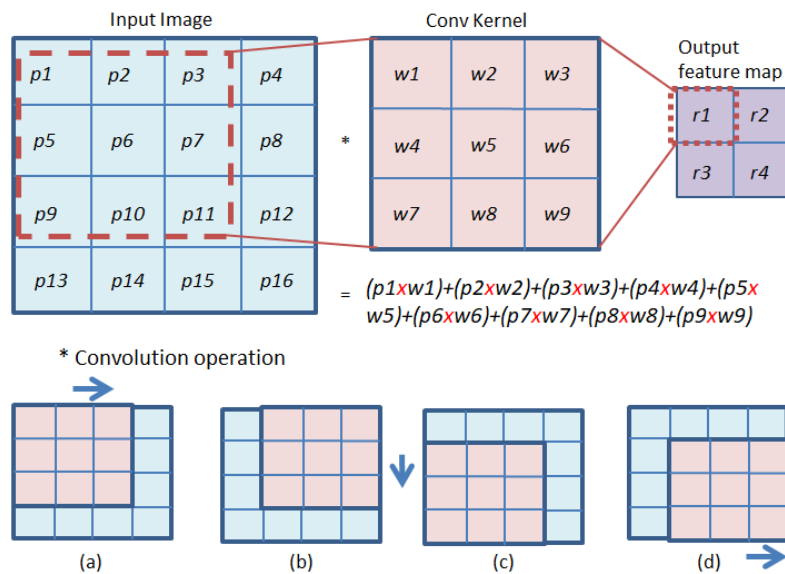
Chapter 3

Foundations

3.0.1 Fundamental Architectural Layers

3.0.1.1 Convolution Layers :

Below figure shows the convolution operation on a simple representative image of 4x4 pixels and a convolution kernel of 3x3, resulting into a output feature map of 2x2.



Above diagram shows the convolution operation in action.

3.0.1.2 Dense Layer

Dense Layer is one of the layers in Deep Learning Architecture. In any neural network, a dense layer is a layer that is deeply connected with its

preceding layer which means the neurons of the layer are connected to every neuron of its preceding layer. This layer is the most commonly used layer in artificial neural network networks.

We have three main attributes: activation function, weight matrix, and bias vector. Using these attributes a dense layer operation can be represented as:

$$\text{Output} = \text{activation}(\text{dot}(\text{input}, \text{kernel}) + \text{bias})$$

3.0.1.3 Deconvolution Layer

Deconvolution is a quantitative approach that uses the picture as an estimate of the real specimen intensity and conducts the mathematical inverse of the imaging process to generate an improved estimate of the image intensity using a formula for the point spread function.

3.0.1.4 Batch Normalization Layer

Layer that normalizes its inputs. Batch normalization applies a transformation that maintains the mean output close to 0 and the output standard deviation close to 1. Importantly, batch normalization works differently during training and during inference.

During training (i.e. when using `fit()` or when calling the layer/model with the argument `training=True`), the layer normalizes its output using the mean and standard deviation of the current batch of inputs.

During inference (i.e. when using `evaluate()` or `predict()` or when calling the layer/model with the argument `training=False` (which is the default), the layer normalizes its output using a moving average of the mean and standard deviation of the batches it has seen during training.

3.0.1.5 Pooling Layer

The purpose of the pooling layers is to reduce the dimensions of the hidden layer by combining the outputs of neuron clusters at the previous layer into a single neuron in the next layer.

The pooling layer summarises the features present in a region of the feature map generated by a convolution layer. So, further operations are performed on summarised features instead of precisely positioned features generated by the convolution layer. This makes the model more robust to variations in the position of the features in the input image.

The pooling operation involves sliding a two-dimensional filter over each channel of feature map and summarising the features lying within the region covered by the filter.

For a feature map having dimensions $nh \times nw \times nc$, the dimensions of output obtained after a pooling layer is :

$$(nh - f + 1) / s \times (nw - f + 1) / s \times nc$$

nh - height of feature map

nw - width of feature map

nc - number of channels in the feature map

f - size of filter

s - stride length

3.0.1.6 Dropout Layer

Dropout is one of the important concept in the machine learning. It is used to fix the over-fitting issue. Input data may have some of the unwanted data, usually called as Noise. Dropout will try to remove the noise data and thus prevent the model from over-fitting.

3.0.1.7 Flatten Layer

Flattening is converting the data into a 1-dimensional array for inputting it to the next layer. We flatten the output of the convolutional layers to create a single long feature vector. And it is connected to the final classification model, which is called a fully-connected layer. It is often used after pooling. Dense Layer can only take 1D data as input hence we have to flatten the input before it is passed to Dense layer.

3.0.1.8 Activation Layer

Activation functions are a critical part of the design of a neural network.

Activation functions are a key part of neural network design. The modern default activation function for hidden layers is the ReLU function. The activation function for output layers depends on the type of prediction problem.

Activation functions are used to introduce non-linearity . For ex in house price prediction using deep learning

Activation for Hidden Layers: Typically, a differentiable nonlinear activation function is used in the hidden layers of a neural network. This allows the model to learn more complex functions than a network trained using a linear activation function.

There are perhaps three activation functions you may want to consider for use in hidden layers; they are:

Rectified Linear Activation (ReLU) Logistic (Sigmoid) Hyperbolic Tangent (Tanh)

Activation for Output Layers: The output layer is the layer in a neural network model that directly outputs a prediction.

All feed-forward neural network models have an output layer.

There are perhaps three activation functions you may want to consider for use in the output layer; they are:

Linear Logistic (Sigmoid) Softmax

3.0.1.9 Recurrent Layer

Recurrent layer is used to create recurrent neural networks. It can also be used for feed forward layers but the input shape is specific (batch size, sequence length, num inputs). The different types of recurrent layers are : .

Custom Recurrent Layer

. Recurrent Layer

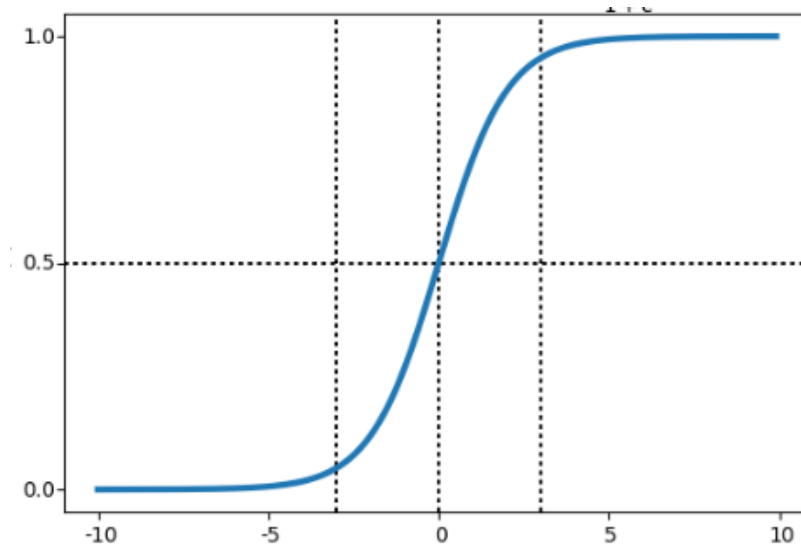
3.0.2 Activation Functions:

Activation functions help the neural networks to use important information while suppressing irrelevant data points. An Activation Function decides whether a neuron should be activated or not. This means that it will decide whether the neuron's input to the network is important or not in the process of prediction using simpler mathematical operations. The role of the Activation Function is to derive output from a set of input values fed to a node (or a layer). The choice of activation functions is most crucial in any deep neural network. Recently lots of activation functions have been introduced like Leaky ReLU, Swish, Mish, etc. Each Activation Function has played a vital role during the course of YOLO evolution, however recently found activation functions like Swish and Mish have helped to improve the performance of the Models.

3.0.2.1 Sigmoid

The main reason why we use sigmoid function is because it exists between (0 to 1). Therefore, it is especially used for models where we must predict the probability as an output, since probability of anything exists only between the range of 0 and 1, sigmoid is the right choice. Formula for Sigmoid Function is

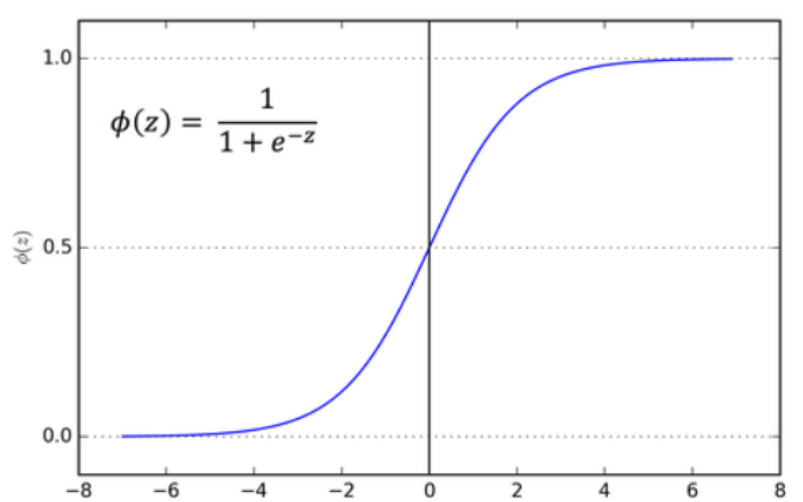
$$S(x) = 1/(1 + e^{-x})$$



3.0.2.2 Softmax

Softmax is a activation function that converts a array of numbers into a array of probabilities, where the probabilities of each value are proportional to the relative scale of each value in the array. Softmax is commonly used in machine learning for logistic regression problems where the highest probability of occurrence decides the class of the output given by the model.

Formula and graph for Softmax Function is :

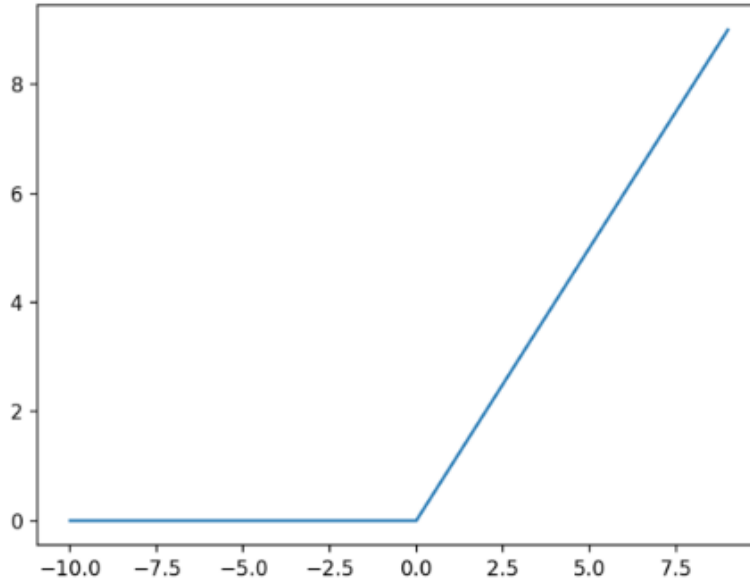


3.0.2.3 Relu

RELU is a activation function which linearly increases as x increases in the positive direction, and remains zero for any negative value of x . Linearity in the positive dimension helps to prevent non-saturation of gradients which is prevalent in sigmoid activation function and it becomes zero for the negative side of the real line.

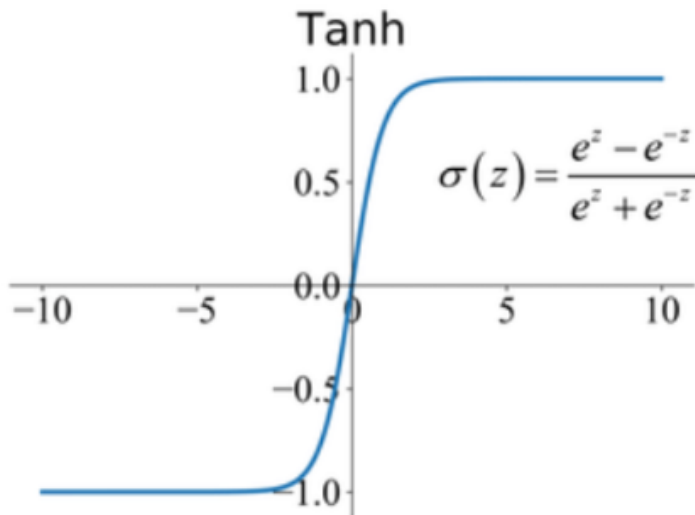
Formula and graph for RELU is

$$f(x) = \max(0, x)$$



3.0.2.4 Tanh

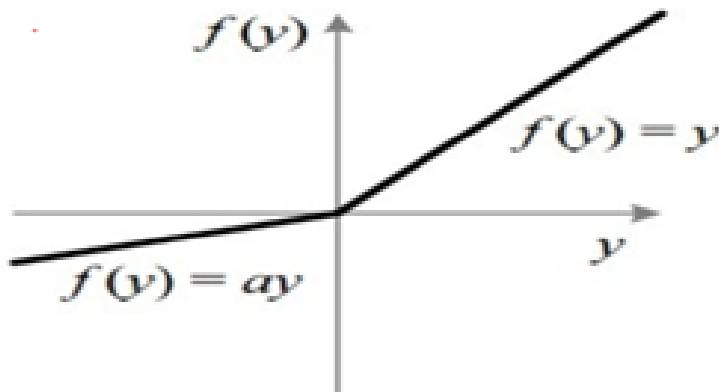
Tanh Activation is an activation function used for neural networks. The tanh function is better than sigmoid function as it gives enhanced performance for multi-layer neural networks. But still it doesn't solve the vanishing gradient problem that we normally experience in sigmoid activation function, Relu is a good alternative to solve this problem. Formula and graph for tanh is



3.0.2.5 LeakyRelu

Leaky Rectified Linear Unit, or Leaky ReLU, is a type of activation function based on a ReLU, but it has a small slope for negative values instead of a flat slope. The slope coefficient is determined before training, i.e. it is not learnt during training. This type of activation function is popular in tasks where we may suffer from sparse gradients, for example training generative adversarial networks. Formula for Leaky Relu is as follows :

$$f(x) = \max(0.01 \cdot x, x)$$



3.0.2.6 ParametricRelu

Parametric ReLU (PReLU) is a modified leaky ReLU that, where the traditional slope values (e.g. 0.01) makes it a parameter for the neural network to figure out when x takes negative values in real life. This has the advantage of overcoming “dying ReLU” problem, as it doesn’t have zero-slope parts.

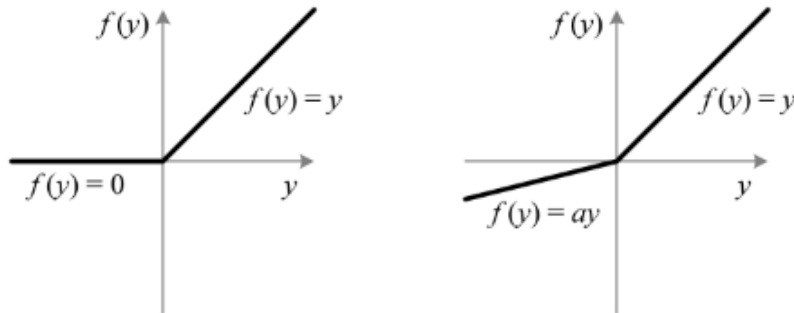
Formula and graph for PRelu is :

$$f(y_i) = y_i$$

$$y_i \geq 0$$

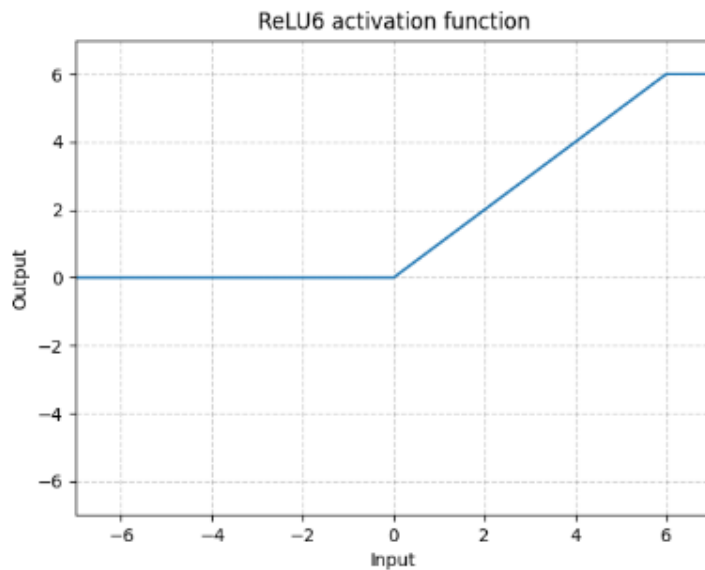
$$f(y_i) = a_i y_i$$

$$y_i \leq 0$$



3.0.2.7 RELU6

[49] ReLU6 is a modification of the rectified linear unit where we limit the activation to a maximum size of . This is due to increased robustness when used with low-precision computation.



3.0.2.8 SELU

[50] Scaled Exponential Linear Units, or SELUs, are activation functions that induce self-normalizing properties.

The SELU activation function is given by

$$f(x) = \lambda x \text{ if } x \geq 0$$

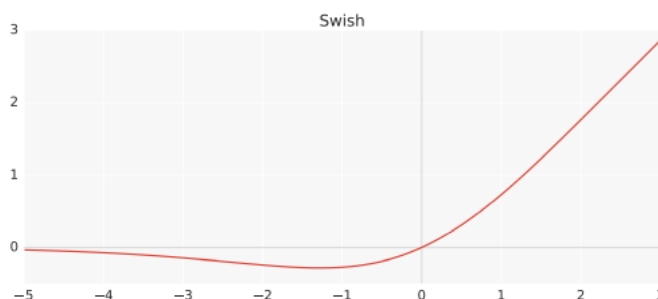
$$f(x) = \lambda \alpha (\exp(x) - 1) \text{ if } x < 0$$

3.0.2.9 Swish

[51] Google brain team published a paper on a new activation function called Swish in the year 2017 named Swish as Neural Networks Activation Function. This activation function is a better alternative to ReLU. ReLU and Leaky ReLU are good activation functions, however, according to this paper experiments revealed that this new activation function overperforms ReLU for deeper networks. The Swish Function is defined as follows :

$$f(x) = x \cdot \text{sigmoid}(x)$$

and the Swish graph looks as follows



This Researcher has compared the performance of ReLU, Leaky ReLU and Swish using multiple datasets and found Swish is either equivalent at networks having 40 or less layers and it outperforms for above 40 layers. The performance has been consistent. In this paper Research Scholar has evaluated Swish and baselined the results with other activation functions first using CIFAR-10 and CIFAR-100 datasets. Models were developed using ReLU function as first and replaced with different activation functions. Same hyperparameters were used in while implementing different activation functions, such as optimizing using SGD with momentum.

Model	ResNet	WRN	DenseNet
LReLU	94.2	95.6	94.7
PReLU	94.1	95.1	94.5
Softplus	94.6	94.9	94.7
ELU	94.1	94.1	94.4
SELU	93.0	93.2	93.9
ReLU	93.8	95.3	94.8
Swish	94.7	95.5	94.8

Model	ResNet	WRN	DenseNet
LReLU	74.2	78.0	83.3
PReLU	74.5	77.3	81.5
Softplus	76.0	78.4	83.7
ELU	75.0	76.0	80.6
SELU	73.2	74.3	80.8
ReLU	74.2	77.8	83.7
Swish	75.1	78.5	83.8

Swish has the properties of one-sided boundedness at zero, smoothness, and non-monotonicity, which may play a role in the observed efficacy. To clear the concept of monotonicity - Monotonic means something that does not vary or change. Non-Monotonic means something which can vary according to the situation or condition. Above results show that ReLU can be easily replaced by Swish to get better results consistently across the domains. Models have to be tweaked when we work on the specific data/domain to get additional gains by trying with different hyperparameters. It is recommended to design the models which are specifically designed with Swish as activation function. The simplicity of Swish and its similarity to ReLU makes the coding job very simple.

3.0.2.10 Mish

[52] Dr. Mishra's group from KIIT Bhubaneswar has proposed a new self regularized activation function inspired by the self gating property of

Swish – Mish. It's mathematical function is defined as

$f(x) = x \tanh(\text{softplus}(x))$.

The softplus function is a smooth approximation to the ReLU activation function, and is sometimes used in the neural networks in place of ReLU.

$$\text{softplus}(x) = \log(1 + e^x)$$

As shown below, Swish and Mish are closely related activation functions. Mish has been validated against several well known benchmarks like ImageNet-1k and it was found that Mish outperformed Leaky ReLU on YOLOv4.

3.0.3 Evaluation Metrics

Evaluation metrics are used to measure the behavior, activities, and performance of the machine learning model. Evaluation metrics become essential when we need to compare two separate models for a given task so that we can understand their efficiency and effectiveness. It is very important to use multiple valuation metrics to evaluate a model. Because a model may perform well using one measurement, but perform poorly using another metric. Evaluation metric can be broadly classified by the type of problem the model is addressing.

3.0.4 CLASSIFICATION METRICS

When the prediction of a classification model is compared by the ground truth and then each instance gets categorized into four categories, true positive, true negative, false positive and false negative. Then we present the distribution in a form of table because the confusion matrix, we can calculate many evaluation metrics in the confusion matrix such as precision, recall, accuracy and F1- score.

	Actually Positive (1)	Actually Negative (0)
Predicted Positive (1)	True Positives (TPs)	False Positives (FPs)
Predicted Negative (0)	False Negatives (FNs)	True Negatives (TNs)

Figure 3.1: Confusion Matrix

3.0.4.1 Accuracy

Accuracy Score is an evaluation metric that measures the number of correct predictions a model made relative to the total number of predictions

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (3.1)$$

made.

3.0.4.2 Precision

Precision is defined as the ratio of correctly classified positive samples (True Positive) to a total number of classified positive samples (either correctly or incorrectly)

$$Precision = \frac{TP}{TP + FP} \quad (3.2)$$

3.0.4.3 Recall

Recall is calculated as the ratio of the number of positive samples correctly classified as positive to the total number of positive samples. The recall measures the model's ability to detect positive samples.

$$Recall = \frac{TP}{TP + FN} \quad (3.3)$$

3.0.4.4 F1-score

F1-score combines the precision and recall of a classifier into a single metric by taking their harmonic mean. It is primarily used to compare the performance of two classifiers. If classifier A has a higher recall, and classifier B has higher precision. In this case, the F1-scores for both the classifiers can be used to determine which one produces better results.

$$F1 - Score = \frac{2TP}{2TP + FP + FN}$$

3.0.4.5 AUC-ROC

AUC ROC Curve is a performance measurement for the classification problems at various threshold settings. ROC curve also known as Receiver Operating Curve is a measure of classifier performance. The ROC curve shows the rate of true positives in comparison to the false alarm rate. Hence the emphasis on sensitivity. ROC can be maximum of 100 percent if there are no false positives. The area under the AUC curve represents the degree of separability. It indicates how well the model can distinguish between categories. The higher the AUC, the better the model is at predicting zero categories as zero and one categories as one.

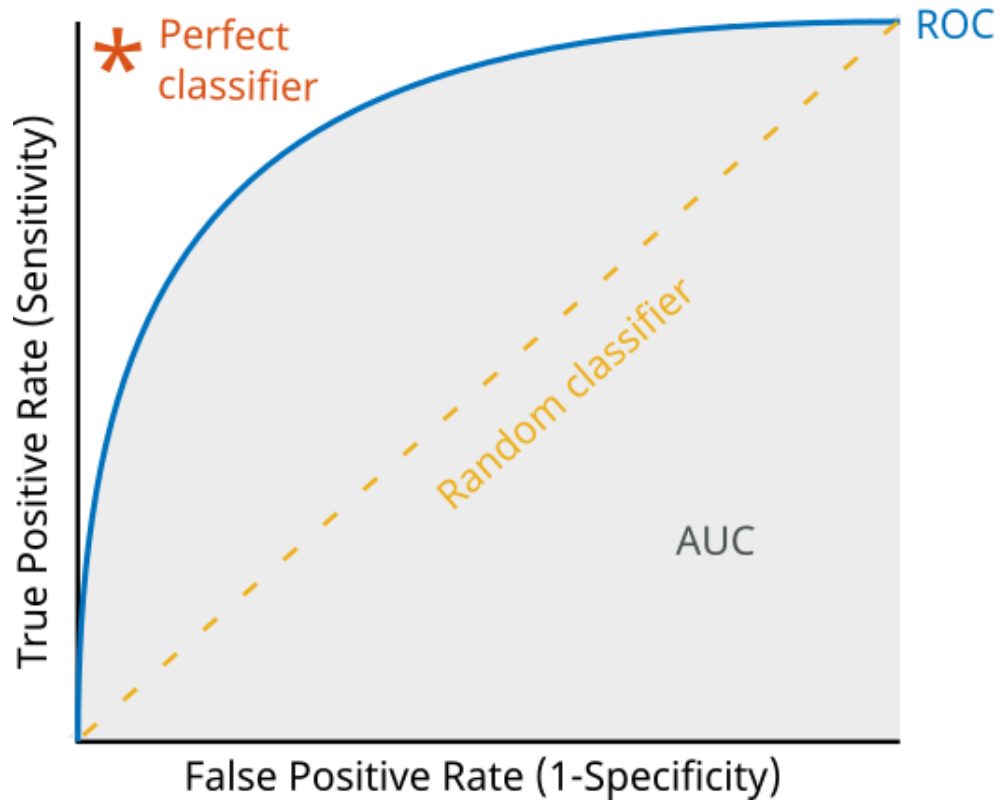


Figure 3.2: AUC-ROC Curve

3.0.5 REGRESSION METRICS

If the model is outputting the values which are continuous in nature then regression evaluation metrics are used

3.0.5.1 Mean Absolute Error -MAE

MAE is the mean of the absolute values of the individual prediction errors on over all instances in the test set.

$$MAE = \frac{|y_i - y_p|}{n} \quad (3.1)$$

y_i = actualvalue

y_p = predictedvalue

n = numberofobservations

3.0.5.2 Root Mean Squared Error-RMSE

RMSE[53] is the square root of the average squared error in the predicted y_i values against actual x_i values

$$rmse = \sqrt{\left(\frac{1}{n}\right) \sum_{i=1}^n (y_i - x_i)^2} \quad (3.2)$$

RMSE is the overall accuracy of the model and is a basis for comparing it to other models and is mainly used in regression models

3.0.6 COMPUTER VISION METRICS

Mohamed Elgendy [54] in his book "Deep Learning for Vision Systems" recommends 1) FPS (frame-per-second) to measure the network detection speed, and 2) mAP (mean Average Precision) to measure the network precision as two main metrics to evaluate the performance of an object detector.

3.0.6.1 Frames per second (FPS)

FPS (Frame Per Second) defines how fast your object detection model process your video and generates the desired output.

3.0.6.2 Intersection Over Union

IOU is used to measure the overlap between the ground truth and predicted bounding boxes. In the numerator we have intersection of both the boxes and the overlap between the boxes or union in the denominator. If there is perfect overlap between the ground truth and predicted bounding box we get a score close to 1. In practice sensitivity of the bounding box is set between 0.7 to 0.5 minimum to enable prediction of objects in images.

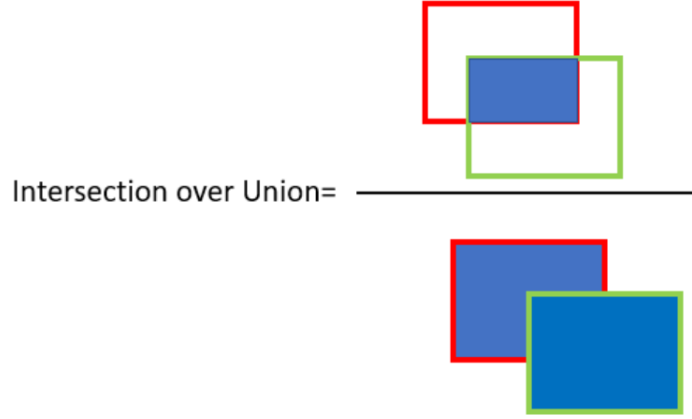


Figure 3.3: Intersection over Union

Red - ground truth bounding box

Green - predicted bounding box

IoU indicates how well predicted and the ground truth bounding box overlap. Depending on set threshold value for the IoU to determine if the object detection is valid or not.

3.0.6.3 Mean Average Precision (mAP)

mAP is the Average Precision over multiple IOU thresholds.

$$mAP = \frac{1}{N} \sum_{i=1}^N AP_i \quad (3.3)$$

For COCO dataset, mAP is calculated as $AP@[.5:.95]$ corresponding to the average AP for IoU from 0.5 to 0.95 with a step size of 0.05. For the COCO competition, AP is the average over 9 IoU levels on 80 categories ($AP@[.50:.05:.95]$: start from 0.5 to 0.95 with a step size of 0.05).

The following 12 metrics are used for characterizing the performance of an object detector on COCO:

Average Precision (AP):

AP % AP at IoU=.50:.95 (primary challenge metric)
 AP_{IoU=.50} % AP at IoU=.50 (PASCAL VOC metric)
 AP_{IoU=.75} % AP at IoU=.75 (strict metric)

AP Across Scales:

AP_{small} % AP for small objects: area < 32²
 AP_{medium} % AP for medium objects: 32² < area < 96²
 AP_{large} % AP for large objects: area > 96²

Average Recall (AR):

AR_{max=1} % AR given 1 detection per image
 AR_{max=10} % AR given 10 detections per image
 AR_{max=100} % AR given 100 detections per image

AR Across Scales:

AR_{small} % AR for small objects: area < 32²
 AR_{medium} % AR for medium objects: 32² < area < 96²
 AR_{large} % AR for large objects: area > 96²

Figure 3.4: COCO Metrics

Image Source : <https://cocodataset.org/detection-eval>

3.0.7 Loss Functions

A Loss function in Machine Learning is a measure of how accurately a built model can predict the expected outcome. In other words, how close is the prediction to the reality. A loss function generally takes two input parameters 'The predicted value by the model and the actual value as per the data'.

A **higher loss** indicates that the model is performing poorly, and a low loss value indicates the model is performing close to the reality.

Selection of a correct loss function is very critical for training a precise model as certain loss functions have certain properties which help the model to be trained better. For example, some may put higher weight age on outliers while others on majority

Below are the list of most important loss functions used in Machine Learning

3.0.7.1 MSE

In Statistics, Mean Squared Error (MSE) is defined as Mean or Average of the square of the difference between actual and estimated values.

Simple Understanding of Mean Squared Error using made up ice-cream sales data in lakhs for a year

Month	Actual Sales	Forecasted Sales	Error	Squared Error
Jan	42	38	4	16
Feb	45	48	-3	9
Mar	70	78	-8	64
Apr	100	105	-5	25
May	125	140	-15	225
Jun	80	75	5	25
Jul	65	62	3	9
Aug	45	38	7	49
Sep	32	28	4	16
Oct	30	28	2	4
Nov	28	25	3	9
Dec	25	20	5	25
Sum				476
MSE				39.66666667

Sum/No of months

From the above example, we can observe the following.

1. As forecasted values can be less than or more than actual values, a simple sum of difference can be zero. This can lead to a false interpretation that forecast is accurate
2. As we take a square, all errors are positive, and mean is positive indicating there is some difference in estimates and actual. Lower mean indicates forecast is closer to actual.
3. All errors in the above example are in the range of 0 to 10 except May, which is -15. As we square it, the difference between this and other squares

increases. And this single high value leads to higher mean. So MSE is influenced by large deviators or outliers.

As this can indicate how close a forecast or estimate is to the actual value, this can be used as a measure to evaluate models in Data Science.

$$MSE = \frac{1}{n} \sum \left(\underbrace{y - \hat{y}}_{\substack{\text{The square of the difference} \\ \text{between actual and} \\ \text{predicted}}} \right)^2$$

3.0.7.2 MAE

In Statistics, Mean Absolute error is the sum of the absolute difference between actual and predicted values.

The formula for MAE :

$$MAE = \frac{\sum_{i=1}^n |y_i - x_i|}{n}$$

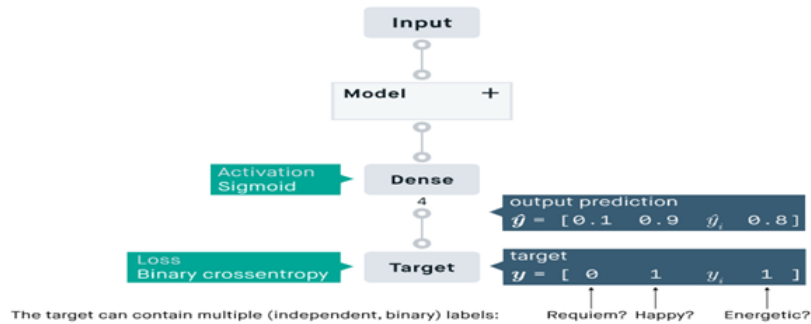
For the above example, if Mean Absolute Value (MAE) is introduced the table will look as below

Month	Actual Sales	Forecasted Sales	Error	Squared Error	MAE
Jan	42	38	4	16	4
Feb	45	48	-3	9	3
Mar	70	78	-8	64	8
Apr	100	105	-5	25	5
May	125	140	-15	225	15
Jun	80	75	5	25	5
Jul	65	62	3	9	3
Aug	45	38	7	49	7
Sep	32	28	4	16	4
Oct	30	28	2	4	2
Nov	28	25	3	9	3
Dec	25	20	5	25	5

3.0.7.3 Binary Cross-Entropy

Binary crossentropy is a loss function that is used in binary classification tasks. These are tasks that answer a question with only two choices (yes or

no, A or B, 0 or 1, left or right). Several independent such questions can be answered at the same time. Formally, this loss is equal to the average of the categorical crossentropy loss on many two-category tasks.



Formula is

$$\text{Loss} = -\frac{1}{\text{output size}} \sum_{i=1}^{\text{output size}} y_i \cdot \log \hat{y}_i + (1 - y_i) \cdot \log (1 - \hat{y}_i)$$

Calculation Methodology :

Entropy of data set minus (P1*log2(P1)+P2*log2(P2))

Home Owner	Income	Credit Score	Default	Predicted
Home Owner	Low	Below Avg	yes	yes
Home Owner	Medium	Avg	yes	yes
Not a Home Owner	High	Avg	no	no
Home Owner	Low	Avg	no	yes
Not a Home Owner	Medium	Below Avg	yes	yes
Home Owner	Medium	Below Avg	no	no
Home Owner	High	High	yes	no
Home Owner	Low	High	no	no
Home Owner	Medium	Avg	yes	no
Not a Home Owner	Medium	Avg	yes	yes
Not a Home Owner	Medium	Below Avg	no	no
Not a Home Owner	High	Below Avg	no	no
Home Owner	High	High	no	no
Home Owner	Low	Avg	yes	no
Home Owner	Medium	Avg	no	no

Count of Default	Default =		
	no	yes	Grand Total
Total	53.33%	46.67%	100.00%

Credit Score	Default =		
	no	yes	Grand Total
Avg	42.86%	57.14%	100.00%
Below Avg	60.00%	40.00%	100.00%
High	66.67%	33.33%	100.00%
Grand Total	53.33%	46.67%	100.00%

Entropy
0.996792
More noise as the num is closer to 1

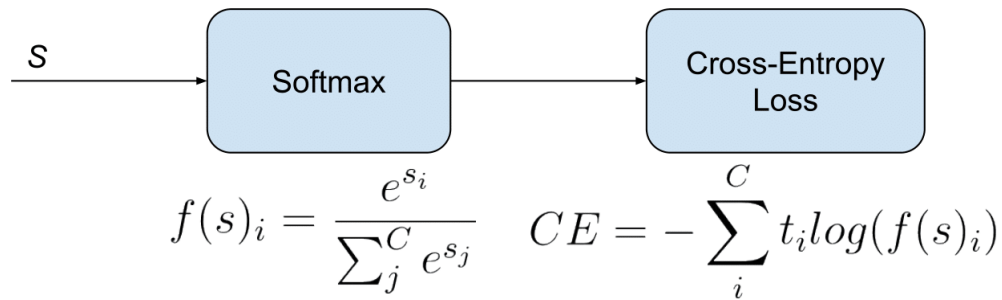
Information Gain
0.398446
Less Noise as the num is not closer to 1

Count of default as yes	7
Total count	15
Prob of default	47%

3.0.7.4 Categorical Cross-Entropy

Categorical Cross Entropy is used as a loss function for multi-class classification model where there are two or more output labels. The output label is assigned one-hot category encoding value in form of 0s and 1. The output label, if present in integer form, is converted into categorical encoding using categorical method.

Also called Softmax Loss. It is a Softmax activation plus a Cross-Entropy loss. If we use this loss, we will train a CNN to output a probability over multiple classes for each image.



3.0.7.5 Balanced Cross Entropy loss

Image detection models has a tendency to fail in distinguishing between hard and easy samples during training phase. Hard samples are those in which the model repeatedly makes wrong prediction, whereas most of the samples are easily and consistently classified right. As a result, Cross-Entropy by definition puts less weight-age because of its population size. Concept called Balanced Cross Entropy loss is introduced to remove this class imbalance in the process. In this loss function a weighting factor for each class was introduced, which is represented by alpha, $[0, 1]$. Alpha could be the inverse class frequency or a hyper-parameter that is determined by cross-validation. The alpha parameter replaces the actual label term in the Cross-Entropy equation.

Formula for Balanced CE is :

$$BalancedCrossEntropy = - \sum_{i=1}^{i=n} \alpha_i \log_b(p_i)$$

3.0.7.6 Focal loss

In-spite of identifying the problem class imbalance and introducing a loss function to address the same we still do not have a solution to differentiate between hard and easy samples. A concept called focal loss [55] was introduced to focus more on the places where the model could go wrong rather where the model predicts correctly. A concept called Down Weighting was introduced to reduce the influence of easy samples on the loss function, re-

sulting in more attention given to hard samples where model predictions are wrong. This technique is implemented by adding a modulating factor.

Formula for Balanced CE is :

$$Focal Loss = - \sum_{i=1}^{i=n} (1 - p_i)^\gamma \log_b(p_i)$$

Where Gamma is the focusing parameter to be tuned using cross-validation. In the case of the misclassified sample, the p_i is small, making the modulating factor approximately or very close to 1. That keeps the loss function unaffected. As a result, it behaves as a Cross-Entropy loss.

As the confidence of the model increases, that is, p_i moves towards 1, modulating factor will tend to 0, thus down-weighting the loss value for well-classified examples. The focusing parameter, Gamma $\gamma = 1$, will re-scale the modulating factor such that the easy examples are down-weighted more than the hard ones, reducing their impact on the loss function.

Following example depicts how concept of down weighting works : Consider predicted probabilities to be 0.9 and 0.6. Considering Gamma = 2, the loss value calculated for 0.9 comes out to be $4.5e-4$ and down-weighted by a factor of 100, for 0.6 to be $3.5e-2$ down-weighted by a factor of 6.25. From the experiments, Gamma = 2 worked the best for the authors of the Focal Loss paper. When Gamma = 0, Focal Loss is equivalent to Cross Entropy.

In practice, we use an Alpha-balanced variant of the focal loss that inherits the characteristics of both the weighing factor Alpha and the focusing parameter Gamma, yielding slightly better accuracy than the non-balanced form.

Focal Loss has proven to be effective at balancing loss by increasing the loss on hard-to-classify classes. However, it tends to produce a vanishing gradient during back-propagation.

3.0.7.7 Varifocal loss

YOLO(v6) models uses varifocal [22] for measuring the loss when classification part is used

Varifocal Loss is a loss function for training a dense object detector to predict the IACS. Unlike the focal loss that deals with positives and negatives equally, Varifocal Loss treats them asymmetrically.

$$\text{VFL}(p, q) = \begin{cases} -q(q \log(p) + (1 - q) \log(1 - p)) & q > 0 \\ -\alpha p^\gamma \log(1 - p) & q = 0, \end{cases}$$

Figure 3.5: Formula for Varifocal Loss

where p is the predicted IoU Aware Classification Score and q is the target IoU score.

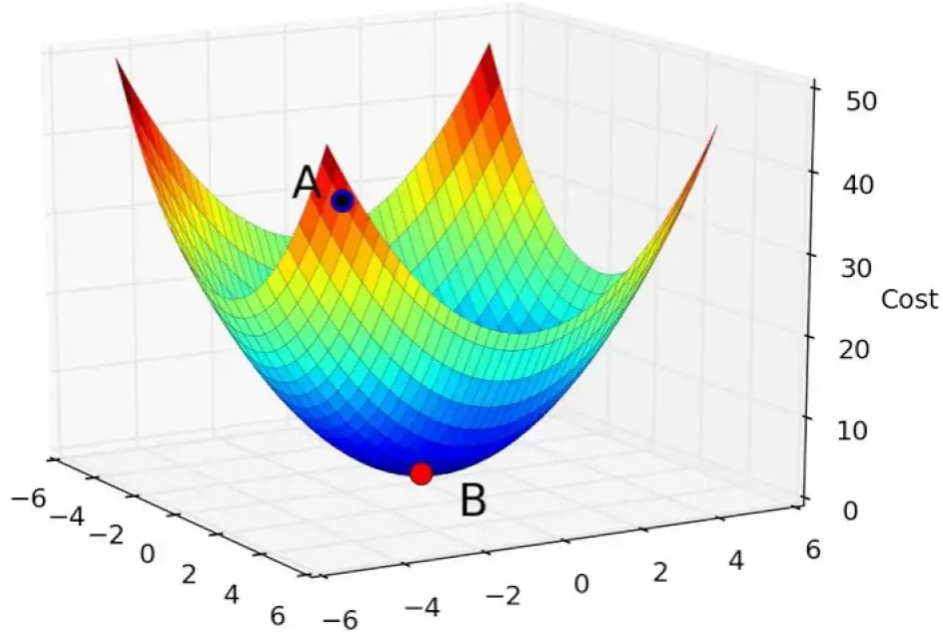
3.0.7.8 Distribution Focal Loss

DFL treats the continuous distribution of box locations as a discretized probability distribution. It is especially helpful in detection when the boundaries of the images are blurred.

DFL (Distribution Focal Loss) along with SIoU or GIoU as box regression loss when we use regression

3.0.8 Optimizers

Usually machine learning and deep neural network training process to get best possible network parameters is iterative and time taking process. An ML/DL engineer may just have to wait for hours, days or even months just for the network to learn best possible parameters. Optimization algorithms in short optimizers are algorithms or methods or processes which helps in reducing training time by adjusting network training parameters to their best possible values.



Gradient descent is the fundamental optimization algorithm serves as backbone of basic training process. The technique itself not so new, existed for many decades in its various forms. It starts from a random initialization points and descends through down through a slope calculated by error differences until it reaches minimal error point for that function, the weights of network are adjusted through a process called as back-propagation.

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \eta_k \nabla f(\mathbf{x}) = \mathbf{x}_k - \eta_k \frac{1}{n} \sum_{i=1}^n \nabla f_i(\mathbf{x}_k)$$

3.0.8.1 SGD

Stochastic Gradient Descent shortly named as SGD is improvised version of Gradient Descent optimization algorithm proposed by Robbins and Monro in 1951. The term S-Stochastic refers to 'Randomness' which is the core of this algorithm. Traditional plain GD works on whole dataset one sample point at a time deterministic steps, while SGD picks one data point randomly and makes quick progress towards confusion region / solution space. Version of SGD replaces actual gradients by gradient of subset of randomly (stochastic) selected batch called 'mini-batch', hence its also called as mini-batch stochastic gradient descent. Thus SGD reduced computations drastically

compared to normal GD. SGD has advantage in terms of efficiency and ease of implementation, however it suffers from additional hyper-parameters and is sensitive to feature scaling. Its great algorithm to make initial progress in training and not caring much about closer to best solution point.

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \eta_k \nabla f_{i(k)}(\mathbf{x}_k)$$

3.0.8.2 AdaGrad

AdaGrad stands for Adaptive Gradient Optimizer. Unlike Fundamental-Gradient-descent and Stochastic-gradient-descent which have fixed rate-of-learning, AdaGrad specializes to adapt learning rates for each weight at each iteration. This technique has great significance as in real-world datasets all features may not uniformly distributed, some may be sparse like bag of words, some may be dense like images with lot of objects. So constant learning rate for all weights in all iterations may not work well. Hence a specialized learning rate helps to converge at optimal minima faster. Advantages of AdaGrad are that it need not have manual tuning of learning rate, more reliable and converges to optimal minima faster. However disadvantages of adagrad are $\alpha(t)$ becomes large for more number of iterations and hence gradient may vanish making convergence slower in such cases.

$$\begin{aligned} \mathbf{w}_t &= \mathbf{w}_{t-1} - \eta'_t \frac{\partial L}{\partial \mathbf{w}(t-1)} \\ \eta'_t &= \frac{\eta}{\sqrt{\alpha_t + \epsilon}} \\ \alpha_t &= \sum_{i=1}^t g_i^2 \\ g_i &= \frac{\partial L}{\partial \mathbf{w}(old)} \end{aligned}$$

3.0.8.3 RMSprop

RMSprop which is short form for Root-Mean-Squared-Propagation is an extension of AdaGrad. Unlike AdaGrad which used entire history of squared gradient, RMSprop uses decaying moving-average of partial-gradients in the adaption of rate of learning. This allows algorithm to concentrate on most recent gradient than early partial gradients. This helps to overcome limitation of AdaGrad. This programatically achieved by adding additional hyper-parameter called 'rho'.

$$s(t+1) = (s(t) * \rho) + (f'(x(t))^2 * (1.0 - \rho))$$

3.0.8.4 Adam

AdaM is short form for Adaptive Momentum optimizer was introduced by Kingma and Jimmy in their paper [56] in 2015. It is a first-order optimization method with gradient-based optimization of random objective-functions. This is based on approximation technique which is adaptive in nature and has moments of lower order. It is efficient in scale of computation and easy to implement. Non-stationary objective functions are benefit by this method. Method suits for noisy as well as sparse-gradient dataset.

Adam is designed to combine advantages of of two former methods AdaGrad [57] and RMSprop. Some of advantages are magnitude of parameter updates are insensitive to rescaling of gradients, learning rate.

3.0.8.5 Nesterov Accelerated Gradient Descent

Nesterov's Accelerated Gradient and Momentum as optimizer is method of solving convex programming problem in Hilbert space. The traditional momentum based approach makes lot of oscillation in and out before it converges into best possible global minima. To avoid such oscillations NAG has proposal to each update in two-steps, first - a partial update, in which 'look ahead' point is obtained, second, then final update (NAG update rule) is done. The first 3 updates of NAG are same as momentum are positive. But 4th update has clear distinction, that there is positive partial update and negative final update. Below update mechanism shows how proportional the current update is - to not only current gradients but also on previous ones. However every step by gamma times their impact is reduced.

$$\begin{aligned}
update_0 &= 0 \\
update_1 &= \gamma \cdot update_0 + \eta \nabla w_1 = \eta \nabla w_1 \\
update_2 &= \gamma \cdot update_1 + \eta \nabla w_2 = \gamma \cdot \eta \nabla w_1 + \eta \nabla w_2 \\
update_3 &= \gamma \cdot update_2 + \eta \nabla w_3 = \gamma(\gamma \cdot \eta \nabla w_1 + \eta \nabla w_2) + \eta \nabla w_3 \\
&= \gamma \cdot update_2 + \eta \nabla w_3 = \gamma^2 \cdot \eta \nabla w_1 + \gamma \cdot \eta \nabla w_2 + \eta \nabla w_3 \\
update_4 &= \gamma \cdot update_3 + \eta \nabla w_4 = \gamma^3 \cdot \eta \nabla w_1 + \gamma^2 \cdot \eta \nabla w_2 + \gamma \cdot \eta \nabla w_3 + \eta \nabla w_4 \\
&\vdots \\
update_t &= \gamma \cdot update_{t-1} + \eta \nabla w_t = \gamma^{t-1} \cdot \eta \nabla w_1 + \gamma^{t-2} \cdot \eta \nabla w_2 + \dots + \eta \nabla w_t
\end{aligned}$$

With momentum:

$$\begin{aligned} \text{update}_t &= \gamma \cdot \text{update}_{t-1} + \eta \nabla w_t \\ w_{t+1} &= w_t - \text{update}_t \end{aligned}$$

With NAG:

$$\begin{aligned} w_{\text{look_ahead}} &= w_t - \gamma \cdot \text{update}_{t-1} \\ \text{update}_t &= \gamma \cdot \text{update}_{t-1} + \eta \nabla w_{\text{look_ahead}} \\ w_{t+1} &= w_t - \text{update}_t \end{aligned}$$

3.0.8.6 Adan

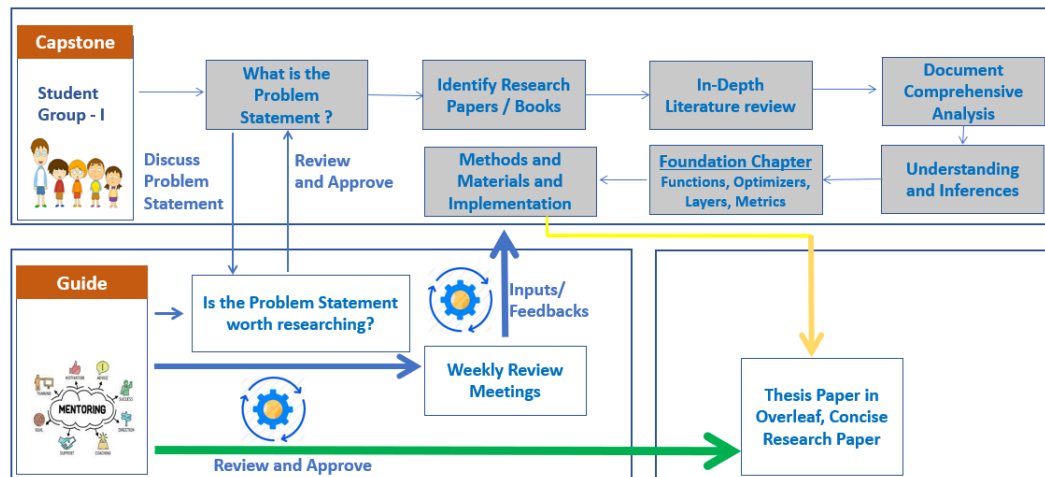
Adaptive gradient algorithms inspired by the concept of moving-average of acceleration of heavy ball. This helps to speed up convergence by estimating accurate gradient moments of first order and second order. In this work, authors propose the ADAPtive Nesterov Impulse Algorithm or called as Adan in short. This helps in effectively accelerate deep neural network training.

Adan builds upon the vanilla Nesterov acceleration to develop a new Nesterov momentum estimation (NME) method. NME avoids the additional computation and memory overhead of calculating the gradient at the extrapolation point. NME applied to approximate the 1st and 2nd order moments of the gradient in adaptive gradient algorithms to accelerate convergence. Authors also demonstrated that Adan identifies a first-order approximate-stationary-point. This point lies within the stochastic gradient complexity on the non-convex stochastic problem. For examples deep learning problems that corresponds to the best-known lower bound.

Chapter 4

Material and Methods

4.0.1 Research Methodology



4.0.2 Datasets

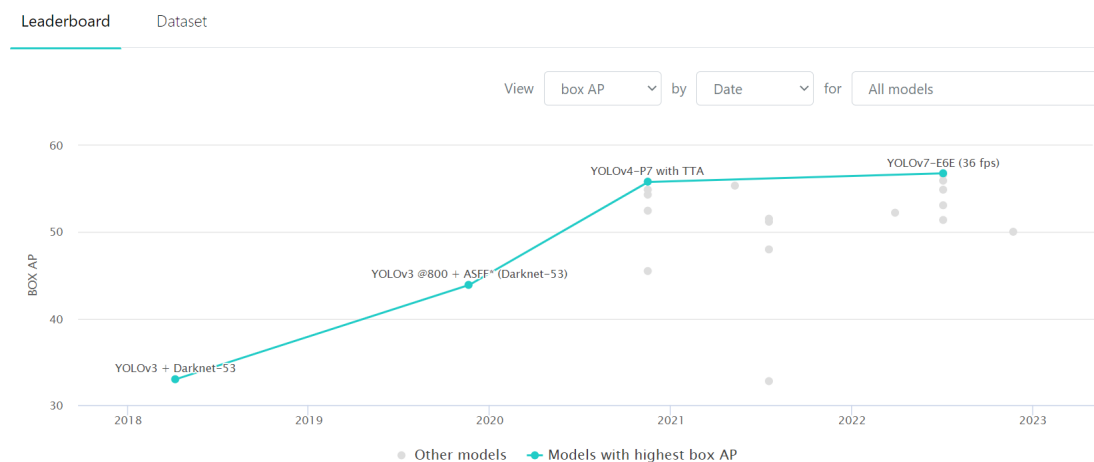
4.0.2.1 Default Dataset (COCO)

Lin et. al. [58] described Microsoft Common Objects in Context (COCO) dataset as gold standard benchmark for evaluating the performance of the most modern computer vision models, including YOLO. COCO contains over 330,000 images. Out of these images more than 200,000 are labeled for 80 object categories. COCO is a collaborative project maintained by computer vision experts from numerous renowned institutions including Google, Caltech, and Georgia Tech. The motivation of the Coco data set is to see how

well we can get models to understand common objects in context, like books, cell phones, people, and cups.

COCO dataset includes 250,000 people with keypoints. This makes an excellent starting point to pre train models on. it allows researchers to train very large models like YOLO and then we can take those pre trained checkpoints from the Coco data, set, crystallize, and model weights and then start on a new task through transfer learning.

Object Detection on COCO test-dev



In the figure, we can see the model performance over time using the box AP as a performance metric.

4.0.2.2 Custom Dataset

For this project we used Chess Data Set from RoboFlow which contained annotations and bounding box classifications to test Yolo V5 and V7

4.0.2.3 Environment

We used Google Colab as well as weights from pre-trained models from RoboFlow. Following code snip shows how Google Colab was used for setting up the environment

▼ Install Dependencies

(Remember to choose GPU in Runtime if not already selected. Runtime → Change Runtime Type → Hardware accelerator → GPU)

```
[ ] # clone YOLOv5 repository
!git clone https://github.com/ultralytics/yolov5 # clone repo
%cd yolov5
!git reset --hard fbe67e465375231474a2ad80a4389efc77ecff99

Cloning into 'yolov5'...
remote: Enumerating objects: 15002, done.
remote: Counting objects: 100% (7/7), done.
remote: Compressing objects: 100% (6/6), done.
remote: Total 15002 (delta 1), reused 3 (delta 1), pack-reused 14995
Receiving objects: 100% (15002/15002), 14.09 MiB | 3.43 MiB/s, done.
Resolving deltas: 100% (10281/10281), done.
/content/yolov5/yolov5
HEAD is now at fbe67e4 Fix `OMP_NUM_THREADS=1` for macOS (#8624)

[ ] # install dependencies as necessary
!pip install -qr requirements.txt # install dependencies (ignore errors)
import torch

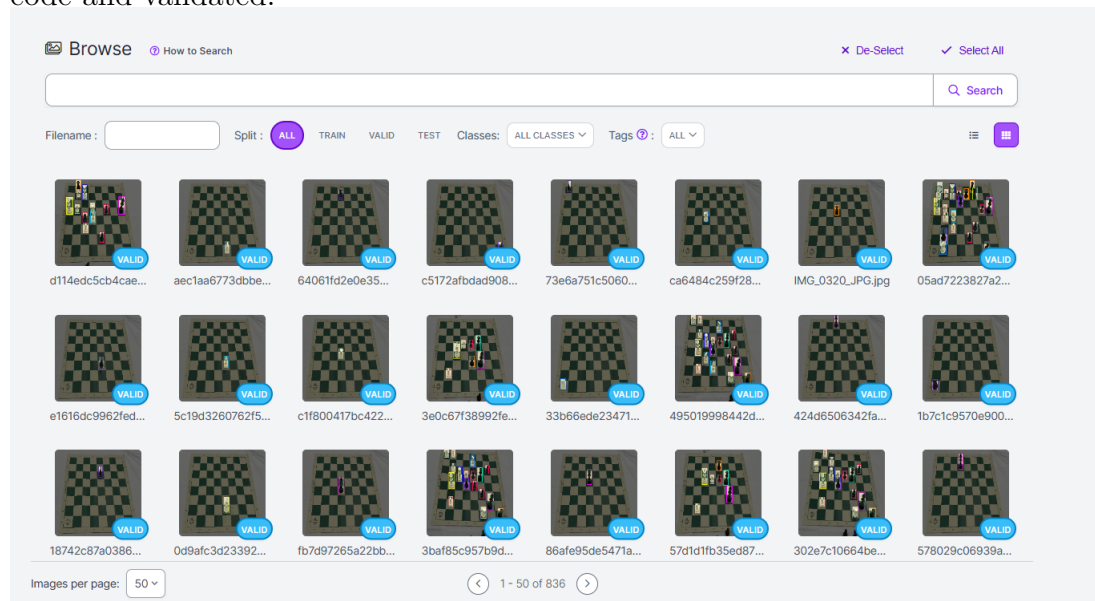
from IPython.display import Image, clear_output # to display images
from utils.downloads import attempt_download # to download models/datasets
```

4.0.3 Data preparation

RoboFlow was used extensively for data preparation, data augmentation and data generation.

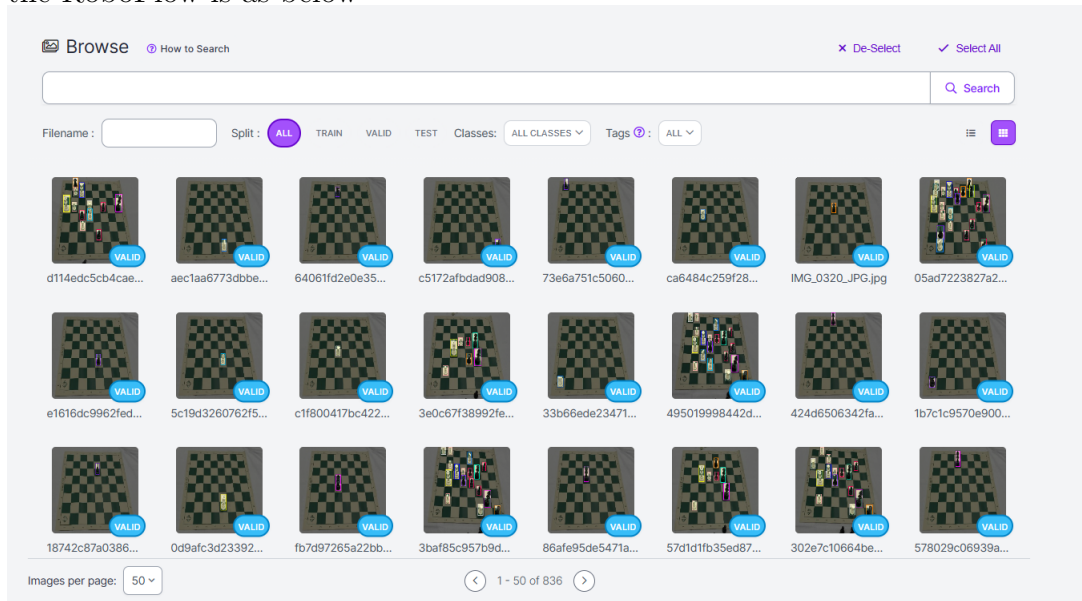
4.0.3.1 Splitting data into training set and validation set

RoboFlow out of box feature of split images for training, testing and validation was used and uploaded into Google drive and it was mounted in Colab code and validated.



4.0.3.2 Preparing the dataset for training

RoboFlow was used for dataset preparation for training. Snap Shot of the RoboFlow is as below



4.0.4 Model Building

Google Colab snap shot for model building is as below
Train Custom YOLOv5 Detector

Next, we'll fire off training!

Here, we are able to pass a number of arguments:

- **img:** define input image size
- **batch:** determine batch size
- **epochs:** define the number of training epochs. (Note: often, 3000+ are common here!)
- **data:** set the path to our yaml file
- **cfg:** specify our model configuration
- **weights:** specify a custom path to weights. (Note: you can download weights from the Ultralytics Google Drive [folder](#))
- **name:** result names
- **nosave:** only save the final checkpoint
- **cache:** cache images for faster training

```
# train yolov5s on custom data for 100 epochs
# time its performance
%%time
%cd /content/yolov5/
!python train.py --img 416 --batch 16 --epochs 100 --data /content/data.yaml --cfg
./models/yolov5s.yaml --weights '' --name yolov5s_results --cache
```

4.0.4.1 YOLO v3

Building YOLOv3 Object Detection project on cloud / google colab as below. We use existing implementation. Lets clone original repository of Darknet from github.

```
# clone darknet repo
!git clone https://github.com/AlexeyAB/darknet
```

As we use GPU version lets change make file to have GPU and OpenCV enabled from our experiment. And let build repo using make command.

```
# change makefile to have GPU and OPENCV enabled
%cd darknet
!sed -i 's/OPENCV=0/OPENCV=1/' Makefile
!sed -i 's/GPU=0/GPU=1/' Makefile
!sed -i 's/CUDNN=0/CUDNN=1/' Makefile
```

Lets use pre-trained weights of YOLOv3 from original website of Darknet.

```
# get yolov3 pretrained coco dataset weights
!wget https://pjreddie.com/media/files/yolov3.weights
```

Lets define some helper functions to show images and upload images to our colab notebook.

```
# define helper functions
def imShow(path):
    import cv2
    import matplotlib.pyplot as plt
    %matplotlib inline

    image = cv2.imread(path)
    height, width = image.shape[:2]
    resized_image = cv2.resize(image, (3*width, 3*height), interpolation = cv2.INTER_CUBIC)

    fig = plt.gcf()
    fig.set_size_inches(18, 10)
    plt.axis("off")
    plt.imshow(cv2.cvtColor(resized_image, cv2.COLOR_BGR2RGB))
    plt.show()

# use this to upload files
def upload():
    from google.colab import files
    uploaded = files.upload()
    for name, data in uploaded.items():
        with open(name, 'wb') as f:
            f.write(data)
        print ('saved file', name)

# use this to download a file
def download(path):
    from google.colab import files
    files.download(path)
```

Darknet is provided with some sample images, and we could use them or we could also include our custom images. For now, let's use images from repo itself.

```
# run darknet detection
!./darknet detect cfg/yolov3.cfg yolov3.weights data/person.jpg
imshow('predictions.jpg')
```

One can think of running detections on more than images outside of the darknet/data/ folder. This step will describe how one can upload local data or Google Drive files to the cloud storage. Then one can run detections on them seamlessly from anywhere.

```
# this creates a symbolic link so that now the path /content/gdrive/My\ Drive/ is equal to /mydrive
!ln -s /content/gdrive/My\ Drive/ /mydrive
!ls /mydrive
```

Below steps show how darknet can be run on custom images stored in a google drive (my image path is /images/plane.jpg)

```
# run darknet command with google drive image (my image path is /images/plane.jpg)
%cd darknet
!./darknet detect cfg/yolov3.cfg yolov3.weights /mydrive/yolov3/Poachers/test_images/images142.jpg
imshow('predictions.jpg')
```

4.0.4.2 YOLO v4

The below snippets of code explains how to run pre-trained YOLO v4 model on COCO dataset using Google Colaboratory :

1. Mount the drive

```
from google.colab import drive
drive.mount('/content/gdrive')
```

2. Clone the Darknet Repo

```
!git clone https://github.com/AlexeyAB/darknet
```

3. Change makefile to have GPU and OPENCV enabled

```
# change makefile to have GPU and OPENCV enabled
%cd darknet
!sed -i 's/OPENCV=0/OPENCV=1/' Makefile
!sed -i 's/GPU=0/GPU=1/' Makefile
!sed -i 's/CUDNN=0/CUDNN=1/' Makefile
!sed -i 's/CUDNN_HALF=0/CUDNN_HALF=1/' Makefile
```

4. Verify CUDA and use make command to build darknet

```
!/usr/local/cuda/bin/nvcc --version
!make
```

5. Download Pre-Trained YOLO v4 weights

```
!wget https://github.com/AlexeyAB/darknet/releases/download/darknet_yolo_v3_optimal/yolov4.weights
```

6. Define the helper functions

```
# define helper functions
def imshow(path):
    import cv2
    import matplotlib.pyplot as plt
    %matplotlib inline

    image = cv2.imread(path)
    height, width = image.shape[:2]
    resized_image = cv2.resize(image, (3*width, 3*height), interpolation = cv2.INTER_CUBIC)

    fig = plt.gcf()
    fig.set_size_inches(18, 10)
    plt.axis("off")
    plt.imshow([cv2.cvtColor(resized_image, cv2.COLOR_BGR2RGB)])
    plt.show()

# use this to upload files
def upload():
    from google.colab import files
    uploaded = files.upload()
    for name, data in uploaded.items():
        with open(name, 'wb') as f:
            f.write(data)
        print('saved file', name)

# use this to download a file
def download(path):
    from google.colab import files
    files.download(path)
```

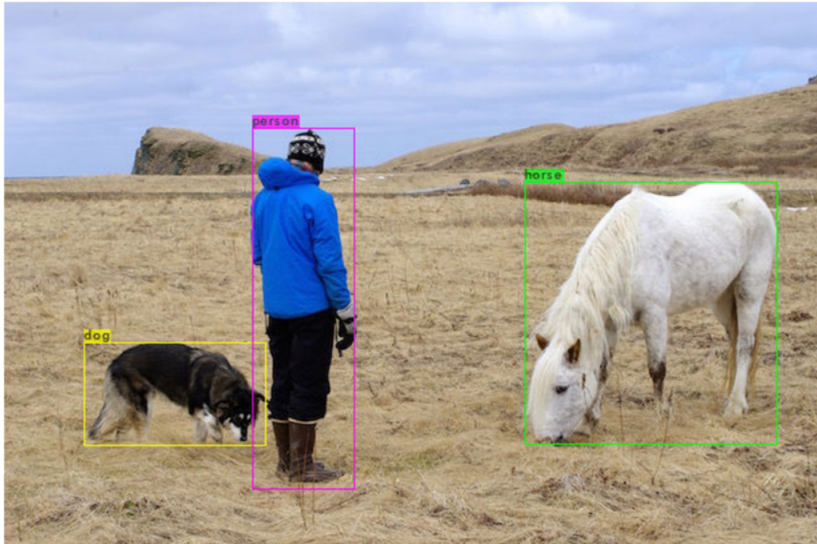
7. Run Your Detections with Darknet and YOLOv4

The object detector can be run using the following command :

```
./darknet detector test cfg/coco.data cfg/yolov4.cfg yolov4.weights data/person.jpg
```

8. show image using our helper function

```
imshow('predictions.jpg')
```



9. To output the bounding box coordinates

```
!./darknet detector test cfg/coco.data cfg/yolov4.cfg yolov4.weights data/person.jpg -ext_output  
imshow('predictions.jpg')
```

dog: 99 percentage (left x: 62 top y: 265 width: 142 height: 80)

person: 100 percentage (left x: 194 top y: 98 width: 78 height: 281)

horse: 98 percentage (left x: 406 top y: 140 width: 196 height: 204)

4.0.4.3 YOLO v7

This section explains how to run pre trained Yolo V7 on cloud GPU using Google Colab environment

Setting up Dependencies

Mounting Google Drive

```
from google.colab import drive
drive.mount('/content/gdrive')
```

Cloning the repo and setting up dependencies

```
%%bash cd /content/gdrive/MyDrive/Capstone/Code
git clone https://github.com/WongKinYiu/yolov7.git cd yolov7
wget https://raw.githubusercontent.com/WongKinYiu/yolov7/u5/requirements.txt
```

```
import os
import sys
sys.path.append('/content/gdrive/MyDrive//Capstone/Code/yolov7')
```

Getting YOLOv7 Models

```
%%bash
wget -P /content/gdrive/MyDrive/yolov7/weights
https://github.com/WongKinYiu/yolov7/releases/download/v0.1/yolov7.pt
wget -P /content/gdrive/MyDrive/yolov7/weights
https://github.com/WongKinYiu/yolov7/releases/download/v0.1/yolov7x.pt
wget -P /content/gdrive/MyDrive/yolov7/weights
https://github.com/WongKinYiu/yolov7/releases/download/v0.1/yolov7-w6.pt
wget -P /content/gdrive/MyDrive/yolov7/weights
https://github.com/WongKinYiu/yolov7/releases/download/v0.1/yolov7-e6.pt
wget -P /content/gdrive/MyDrive/yolov7/weights
https://github.com/WongKinYiu/yolov7
/releases/download/v0.1/yolov7-d6.pt
wget -P /content/gdrive/MyDrive/yolov7/weights
https://github.com/WongKinYiu/yolov7/releases/download/v0.1/yolov7-e6e.pt
```


Helper code for inference

```
# import dependencies
from IPython.display import display, Javascript, Image
from google.colab.output import eval_js
from google.colab.patches import cv2_imshow
from base64 import b64decode, b64encode
import PIL
import io
import html

# function to convert the JavaScript object into an OpenCV image
def js_to_image(js_reply):
    """
    Params:
        js_reply: JavaScript object containing image from webcam
    Returns:
        img: OpenCV BGR image
    """
    # decode base64 image
    image_bytes = b64decode(js_reply.split(',')[1])
    # convert bytes to numpy array
    jpg_as_np = np.frombuffer(image_bytes, dtype=np.uint8)
    # decode numpy array into OpenCV BGR image
    img = cv2.imdecode(jpg_as_np, flags=1)
    return img

# function to convert OpenCV Rectangle bounding box image into base64 byte
# string to be overlaid on video stream
def bbox_to_bytes(bbox_array):
    """
    Params:
        bbox_array: Numpy array (pixels) containing rectangle to overlay
        on video stream.
    Returns:
        bytes: Base64 image byte string
    """
    # convert array into PIL image
    bbox_PIL = PIL.Image.fromarray(bbox_array, 'RGBA')
    iobuf = io.BytesIO()
    # format bbox into png for return
    bbox_PIL.save(iobuf, format='png')
    # format return string
    bbox_bytes = 'data:image/png;base64,{}'.format((str(b64encode(iobuf.getv
alue()), 'utf-8'))))

    return bbox_bytes
```

```

# JavaScript to properly create our live video stream using our webcam as
input
def video_stream():
    js = Javascript('')
    var video;
    var div = null;
    var stream;
    var captureCanvas;
    var imgElement;
    var labelElement;

    var pendingResolve = null;
    var shutdown = false;

    function removeDom() {
        stream.getVideoTracks()[0].stop();
        video.remove();
        div.remove();
        video = null;
        div = null;
        stream = null;
        imgElement = null;
        captureCanvas = null;
        labelElement = null;
    }

    function onAnimationFrame() {
        if (!shutdown) {
            window.requestAnimationFrame(onAnimationFrame);
        }
        if (pendingResolve) {
            var result = '';
            if (!shutdown) {
                captureCanvas.getContext('2d').drawImage(video, 0, 0, 640, 480);
                result = captureCanvas.toDataURL('image/jpeg', 0.8)
            }
            var lp = pendingResolve;
            pendingResolve = null;
            lp(result);
        }
    }

    async function createDom() {
        if (div !== null) {
            return stream;
        }

        div = document.createElement('div');
        div.style.border = '2px solid black';
        div.style.padding = '3px';
        div.style.width = '100%';

```

```

div.style.maxWidth = '600px';
document.body.appendChild(div);

const modelOut = document.createElement('div');
modelOut.innerHTML = "<span>Status:</span>";
labelElement = document.createElement('span');
labelElement.innerText = 'No data';
labelElement.style.fontWeight = 'bold';
modelOut.appendChild(labelElement);
div.appendChild(modelOut);

video = document.createElement('video');
video.style.display = 'block';
video.width = div.clientWidth - 6;
video.setAttribute('playsinline', '');
video.onclick = () => { shutdown = true; };
stream = await navigator.mediaDevices.getUserMedia(
  {video: { facingMode: "environment"}});
div.appendChild(video);

imgElement = document.createElement('img');
imgElement.style.position = 'absolute';
imgElement.style.zIndex = 1;
imgElement.onclick = () => { shutdown = true; };
div.appendChild(imgElement);

const instruction = document.createElement('div');
instruction.innerHTML =
  '<span style="color: red; font-weight: bold;">' +
  'When finished, click here or on the video to stop this demo</span>';
div.appendChild(instruction);
instruction.onclick = () => { shutdown = true; };

video.srcObject = stream;
await video.play();

captureCanvas = document.createElement('canvas');
captureCanvas.width = 640; //video.videoWidth;
captureCanvas.height = 480; //video.videoHeight;
window.requestAnimationFrame(onAnimationFrame);

return stream;
}

async function stream_frame(label, imgData) {
  if (shutdown) {
    removeDom();
    shutdown = false;
    return '';
  }
}

```

```

        var preCreate = Date.now();
        stream = await createDom();

        var preShow = Date.now();
        if (label != "") {
            labelElement.innerHTML = label;
        }

        if (imgData != "") {
            var videoRect = video.getClientRects()[0];
            imgElement.style.top = videoRect.top + "px";
            imgElement.style.left = videoRect.left + "px";
            imgElement.style.width = videoRect.width + "px";
            imgElement.style.height = videoRect.height + "px";
            imgElement.src = imgData;
        }

        var preCapture = Date.now();
        var result = await new Promise(function(resolve, reject) {
            pendingResolve = resolve;
        });
        shutdown = false;

        return { 'create': preShow - preCreate,
                'show': preCapture - preShow,
                'capture': Date.now() - preCapture,
                'img': result };
    })

    display(js)

def video_frame(label, bbox):
    data = eval_js('stream_frame("{} ", "{} ")'.format(label, bbox))
    return data

```

Configuration Parameters

```
classes_to_filter = ['train']  
| #You can give list of classes to filter by name,  
  
opt = {  
    # Path to weights file default weights are for nano model  
    "weights": "weights/yolov7.pt",  
    "yaml"    : "data/coco.yaml",  
    "img-size": 640, # default image size  
    "conf-thres": 0.25, # confidence threshold for inference.  
    "iou-thres" : 0.45, # NMS IoU threshold for inference.  
    "device" : '0', # device to run our model i.e. 0 or 0,1,2,3 or cpu  
    "classes" : classes_to_filter # list of classes to filter or None  
}
```

Inference on single image

```

# Give path of source image
source_image_path = '/content/gdrive/MyDrive/Capstone/Code/yolov7/inferenc
e/images/horses.jpg'
with torch.no_grad():
    weights, imgsz = opt['weights'], opt['img-size']
    set_logging()
    device = select_device(opt['device'])
    half = device.type != 'cpu'
    model = attempt_load(weights, map_location=device) # load FP32 model
    stride = int(model.stride.max()) # model stride
    imgsz = check_img_size(imgsz, s=stride) # check img_size
    if half:
        model.half()

    names = model.module.names if hasattr(model, 'module') else model.names
    colors = [[random.randint(0, 255) for _ in range(3)] for _ in names]
    if device.type != 'cpu':
        model(torch.zeros(1, 3, imgsz, imgsz).to(device).type_as(next(model.pa
rameters()))))

    img0 = cv2.imread(source_image_path)
    img = letterbox(img0, imgsz, stride=stride)[0]
    img = img[:, :, ::-1].transpose(2, 0, 1) # BGR to RGB, to 3x416x416
    img = np.ascontiguousarray(img)
    img = torch.from_numpy(img).to(device)
    img = img.half() if half else img.float() # uint8 to fp16/32
    img /= 255.0 # 0 - 255 to 0.0 - 1.0
    if img.ndimension() == 3:
        img = img.unsqueeze(0)

    # Inference
    t1 = time_synchronized()
    pred = model(img, augment=False)[0]

    # Apply NMS
    classes = None
    if opt['classes']:
        classes = []
        for class_name in opt['classes']:

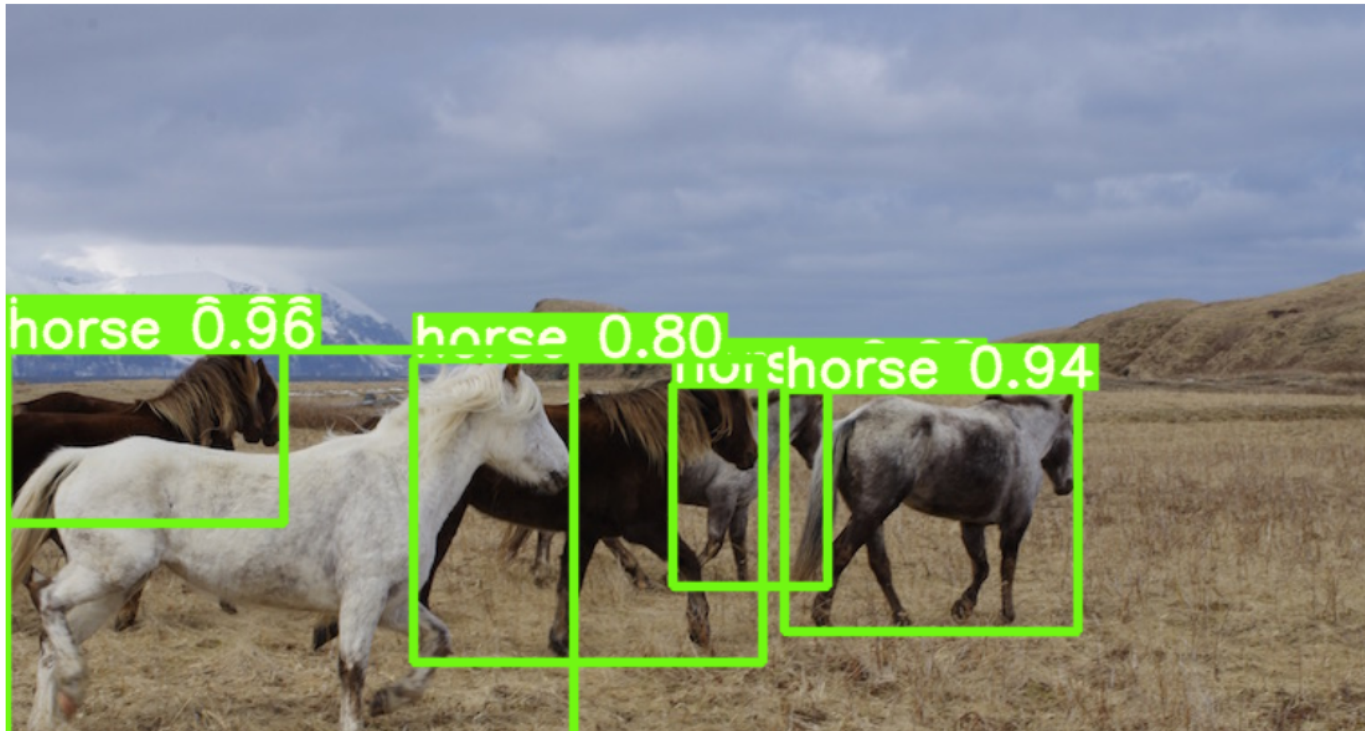
            classes.append(names.index(class_name))

    if classes:
        classes = [i for i in range(len(names)) if i not in classes]

    pred = non_max_suppression(pred, opt['conf-thres'], opt['iou-
thres'], classes= [17], agnostic=False)
    t2 = time_synchronized()
    for i, det in enumerate(pred):
        s = ''
        s += '%gx%g ' % img.shape[2:] # print string
        gn = torch.tensor(img0.shape)[[1, 0, 1, 0]]

```

```
from google.colab.patches import cv2_imshow
cv2_imshow(img0)
```



Inference on Video

Upload video from Local System

```
from google.colab import files
uploaded = files.upload()
```

Download video from Google Drive Link

```
#change URL
!gdown --
fuzzy https://drive.google.com/file/d/16voMoZOiP3Bm0W8Tpu5YotBBpxK_Fs-
o/view?usp=sharing
|
```

Download from any public URL

```
! wget PUBLIC_URL_TO_MP4/AVI_FILE
```

Enter Video Path

```
#give the full path to video, your video will be in the Yolov7 folder  
video_path = '/content/gdrive/MyDrive/Capstone/Code/yolov7/Test.mp4'
```


Run YOLOv7 inference on video

```
# Initializing video object
video = cv2.VideoCapture(video_path)

#Video information
fps = video.get(cv2.CAP_PROP_FPS)
w = int(video.get(cv2.CAP_PROP_FRAME_WIDTH))
h = int(video.get(cv2.CAP_PROP_FRAME_HEIGHT))
nframes = int(video.get(cv2.CAP_PROP_FRAME_COUNT))

# Initializing object for writing video output
output = cv2.VideoWriter('output.mp4', cv2.VideoWriter_fourcc(*'DIVX'), fps
, (w,h))
torch.cuda.empty_cache()
# Initializing model and setting it for inference
with torch.no_grad():
    weights, imgsz = opt['weights'], opt['img-size']
    set_logging()
    device = select_device(opt['device'])
    half = device.type != 'cpu'
    model = attempt_load(weights, map_location=device) # load FP32 model
    stride = int(model.stride.max()) # model stride
    imgsz = check_img_size(imgsz, s=stride) # check img_size
    if half:
        model.half()

    names = model.module.names if hasattr(model, 'module') else model.names
    colors = [[random.randint(0, 255) for _ in range(3)] for _ in names]
    if device.type != 'cpu':
        model(torch.zeros(1, 3, imgsz, imgsz).to(device).type_as(next(model.parameters()))))

    classes = None
    if opt['classes']:
        classes = []
        for class_name in opt['classes']:
            classes.append(names.index(class_name))

    if classes:
        classes = [i for i in range(len(names)) if i not in classes]

    for j in range(nframes):

        ret, img0 = video.read()

        if ret:
            img = letterbox(img0, imgsz, stride=stride)[0]
            img = img[:, :, :-1]
            img = img.transpose(2, 0, 1) # BGR to RGB, to 3x416x416

# Initializing video object
video = cv2.VideoCapture(video_path)
```

Download Inference Video

```
from IPython.display import HTML
from base64 import b64encode
import os

# Input video path
save_path = '/content/gdrive/MyDrive/Capstone/Code/yolov7/output.mp4'

# Compressed video path
compressed_path = "/content/result_compressed.mp4"

os.system(f"ffmpeg -i {save_path} -vcodec libx264 {compressed_path}")

# Show video
mp4 = open(compressed_path, 'rb').read()
data_url = "data:video/mp4;base64," + b64encode(mp4).decode()
HTML("""
<video width=400 controls>
  <source src="%s" type="video/mp4">
</video>
""" % data_url)
```

Webcam Helper Functions

```
# import dependencies
from IPython.display import display, Javascript, Image
from google.colab.output import eval_js
from google.colab.patches import cv2_imshow
from base64 import b64decode, b64encode
import PIL
import io
import html

# function to convert the JavaScript object into an OpenCV image
def js_to_image(js_reply):
    """
    Params:
        js_reply: JavaScript object containing image from webcam
    Returns:
        img: OpenCV BGR image
    """
    # decode base64 image
    image_bytes = b64decode(js_reply.split(',')[1])
    # convert bytes to numpy array
    jpg_as_np = np.frombuffer(image_bytes, dtype=np.uint8)
    # decode numpy array into OpenCV BGR image
    img = cv2.imdecode(jpg_as_np, flags=1)

    return img

# function to convert OpenCV Rectangle bounding box image into base64 byte
# string to be overlayed on video stream
def bbox_to_bytes(bbox_array):
    """
    Params:
        bbox_array: Numpy array (pixels) containing rectangle to overlay
        on video stream.
    Returns:
        bytes: Base64 image byte string
    """
    # convert array into PIL image
    bbox_PIL = PIL.Image.fromarray(bbox_array, 'RGBA')
    iobuf = io.BytesIO()
    # format bbox into png for return
    bbox_PIL.save(iobuf, format='png')
    # format return string
    bbox_bytes = 'data:image/png;base64,{}'.format((str(b64encode(iobuf.getv
alue()), 'utf-8'))

    return bbox_bytes

# JavaScript to properly create our live video stream using our webcam as
# input
def video_stream():
    js = Javascript('''
        var video;
        var div = null;
        var stream;
    ''')
```

```

    modelOut.innerHTML = "<span>Status:</span>";
    labelElement = document.createElement('span');
    labelElement.innerText = 'No data';
    labelElement.style.fontWeight = 'bold';
    modelOut.appendChild(labelElement);
    div.appendChild(modelOut);

    video = document.createElement('video');
    video.style.display = 'block';
    video.width = div.clientWidth - 6;
    video.setAttribute('playsinline', '');
    video.onclick = () => { shutdown = true; };
    stream = await navigator.mediaDevices.getUserMedia(
        {video: { facingMode: "environment" }});
    div.appendChild(video);

    imgElement = document.createElement('img');
    imgElement.style.position = 'absolute';
    imgElement.style.zIndex = 1;
    imgElement.onclick = () => { shutdown = true; };
    div.appendChild(imgElement);

    const instruction = document.createElement('div');
    instruction.innerHTML =
        '<span style="color: red; font-weight: bold;">' +
        'When finished, click here or on the video to stop this demo</sp
an>';
    div.appendChild(instruction);
    instruction.onclick = () => { shutdown = true; };

    video.srcObject = stream;
    await video.play();

    captureCanvas = document.createElement('canvas');
    captureCanvas.width = 640; //video.videoWidth;
    captureCanvas.height = 480; //video.videoHeight;
    window.requestAnimationFrame(onAnimationFrame);

    return stream;
}

async function stream_frame(label, imgData) {
    if (shutdown) {
        removeDom();
        shutdown = false;
        return '';
    }
}

var preCreate = Date.now();
stream = await createDom();

```

```

    var preShow = Date.now();
    if (label !== "") {
        labelElement.innerHTML = label;
    }

    if (imgData !== "") {
        var videoRect = video.getClientRects()[0];
        imgElement.style.top = videoRect.top + "px";
        imgElement.style.left = videoRect.left + "px";
        imgElement.style.width = videoRect.width + "px";
        imgElement.style.height = videoRect.height + "px";
        imgElement.src = imgData;
    }

    var preCapture = Date.now();
    var result = await new Promise(function(resolve, reject) {
        pendingResolve = resolve;
    });
    shutdown = false;

    return { 'create': preShow - preCreate,
            'show': preCapture - preShow,
            'capture': Date.now() - preCapture,
            'img': result };
}

'''

display(js)

def video_frame(label, bbox):
    data = eval_js('stream_frame("{} ", "{} ")'.format(label, bbox))
    return data

```

Inference on webcam

```
# start streaming video from webcam
video_stream()
# label for video
label_html = 'Capturing...'
# initialize bounding box to empty
bbox = ''
count = 0

with torch.no_grad():
    weights, imgsz = opt['weights'], (480, 640)
    set_logging()
    device = select_device(opt['device'])
    half = device.type != 'cpu'
    model = attempt_load(weights, map_location=device) # load FP32 model
    stride = int(model.stride.max()) # model stride

    if half:
        model.half()

    names = model.module.names if hasattr(model, 'module') else model.names
    colors = [[random.randint(0, 255) for _ in range(3)] for _ in names]
    if device.type != 'cpu':
        model(torch.zeros(1, 3, imgsz[0], imgsz[1]).to(device).type_as(next(model.parameters()))))
    classes = None
    if opt['classes']:
        classes = []
        for class_name in opt['classes']:

            classes.append(names.index(class_name))

    if classes:
        classes = [i for i in range(len(names)) if i not in classes]

    while True:
        js_reply = video_frame(label_html, bbox)
        if not js_reply:
            break

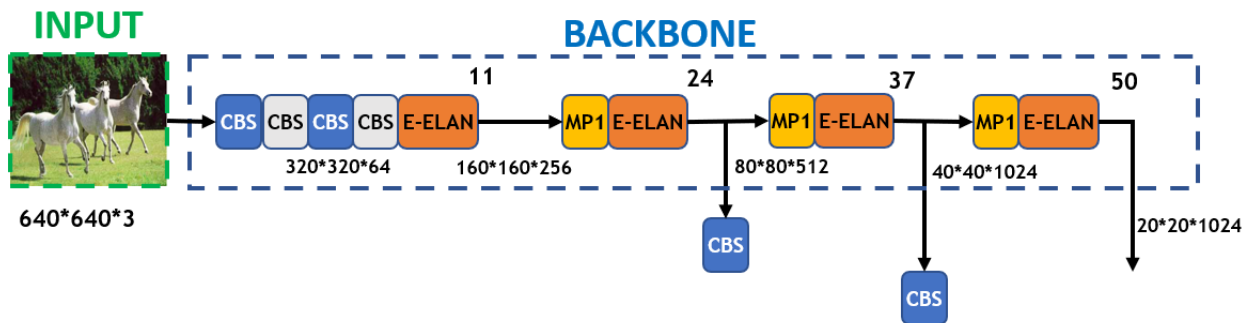
        img0 = js_to_image(js_reply["img"])
        bbox_array = np.zeros([480, 640, 4], dtype=np.uint8)
        img = letterbox(img0, imgsz, stride=stride)[0]
        img = img[:, :, ::-1].transpose(2, 0, 1) # BGR to RGB, to 3x416x416
        img = np.ascontiguousarray(img)
        img = torch.from_numpy(img).to(device)
        img = img.half() if half else img.float() # uint8 to fp16/32
        img /= 255.0 # 0 - 255 to 0.0 - 1.0
        if img.ndimension() == 3:
            img = img.unsqueeze(0)

# Inference
t1 = time_synchronized()
```

Detailed Building Block of Yolo Architecture

Backbone

The backbone consists of 51 layers which takes an input image of 640 x 640 x 3 size and outputs three size images of 80 x 80 x 512 ; 40 x 40 x 1024 ; 20 x 20 x 1024 at the 24th , 37th and 50th layers respectively.



Interpretation of CBS module

CBS is mainly composed of Convolution + Batch Normalization + Activation function which is SiLU

Model(

(model): Sequential(

(0): Conv(

(conv): Conv2d(3, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))

(act): SiLU(inplace=True)

)

(1): Conv(

(conv): Conv2d(32, 64, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1))

(act): SiLU(inplace=True)

)

we can see from the figure that it consists of a Conv layer, which is a convolutional layer, a BN layer, which is a Batch normalization layer, and a Silu layer, which is an activation function. The silu activation function is a variant of the swish activation function, and the formulas of the two are as follows

$$\text{silu}(x) = x \cdot \text{sigmoid}(x)$$

$$\text{swish}(x) = x \cdot \text{sigmoid}(\beta x)$$

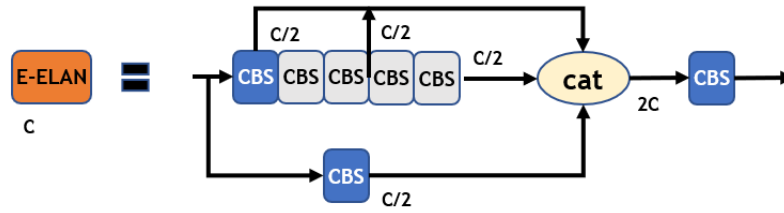
Conv2D $\rightarrow$ *BatchNorm*

$$\begin{aligned} & ((wx + b) - m)/v \\ &= (wx + (b - m))/v \\ &= (w/v)x + (b - m)/v \\ &= w'x + b' \end{aligned}$$

the convolution and batch normalization layer are merged into a single convolution layer

Interpretation of ELAN module

E-ELAN (Extended Efficient Layer Aggregation Network) is the computational block in the YOLOv7 backbone.



```
(8): Conv(
  (conv): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (act): SiLU(inplace=True)
)
(9): Conv(
  (conv): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (act): SiLU(inplace=True)
)
(10): Concat()
(11): Conv(
  (conv): Conv2d(256, 256, kernel_size=(1, 1), stride=(1, 1))
  (act): SiLU(inplace=True)
)
```

The ELAN module is an efficient network structure, which enables the net-

work to learn more features and has stronger robustness by controlling the shortest and longest gradient paths .

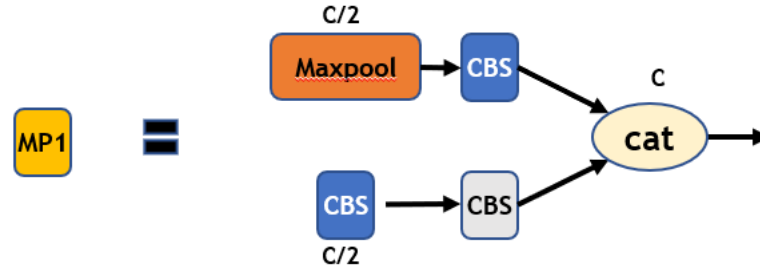
ELAN has two branches.

The first branch is to change the number of channels through a 1x1 convolution.

The second branch is more complicated. It first passes through a 1x1 convolution module to change the number of channels. Then go through four 3x3 convolution modules for feature extraction.

As shown in the figure, finally the four features are superimposed together to obtain the final feature extraction result.

Interpretation of MP module



```
(12): MP(
    (m): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
)
(13): Conv(
    (conv): Conv2d(256, 128, kernel_size=(1, 1), stride=(1, 1))
    (act): SiLU(inplace=True)
)
(14): Conv(
    (conv): Conv2d(256, 128, kernel_size=(1, 1), stride=(1, 1))
    (act): SiLU(inplace=True)
)
(15): Conv(
    (conv): Conv2d(128, 128, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1))
    (act): SiLU(inplace=True)
)
(16): Concat()
```

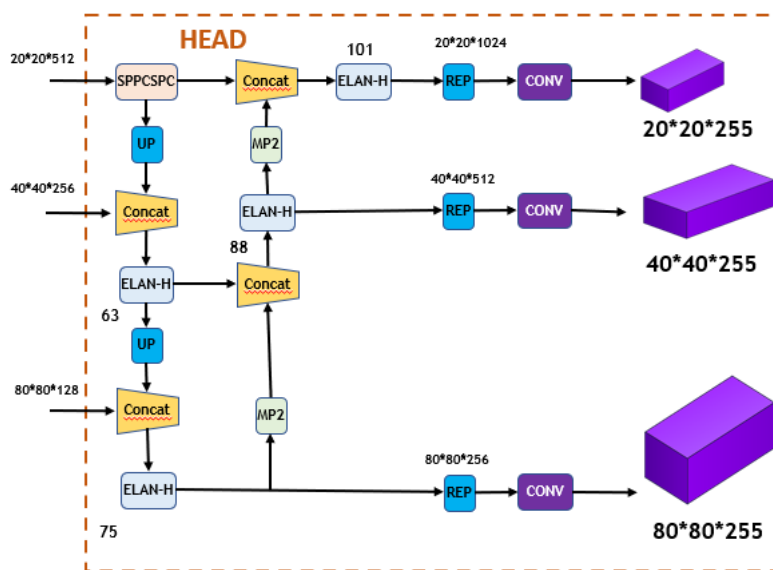
The MP module has two branches, which are used for downsampling . The first branch first passes through a maxpool, which is the maximum pooling.

The function of maximization is down sampling, and then a 1x1 convolution is performed to change the number of channels.

The second branch first passes through a 1x1 convolution to change the number of channels, and then passes through a convolution block with a 3x3 convolution kernel and a step size of 2. This convolution block is also used for down sampling. Finally, the results of the first branch and the second branch are added together to obtain the result of super downsampling

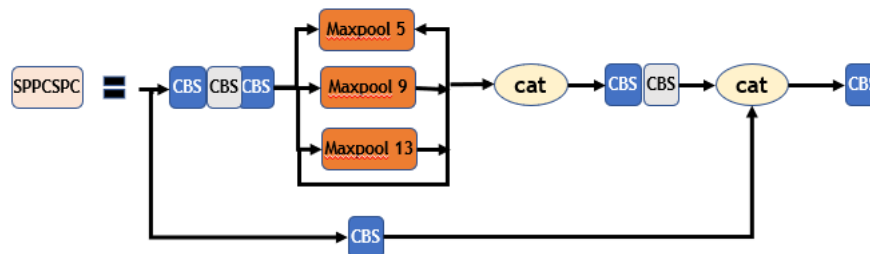
Head

YOLOV7 head is actually a Path Aggregation Feature Pyramid Network (PAFPN) structure, which is the same as the previous YOLOV4 and YOLOV5.



Interpretation of SPPCSPC module

SPPCSPC is a combination of SPP (Spatial Pyramid Pooling Layer) and CSPC (Cross Stage Partial Network)



```

(51): SPPCSPC(
    (cv1): Conv(
        (conv): Conv2d(1024, 512, kernel_size=(1, 1), stride=(1, 1))
        (act): SiLU(inplace=True)
    )
    (cv2): Conv(
        (conv): Conv2d(1024, 512, kernel_size=(1, 1), stride=(1, 1))
        (act): SiLU(inplace=True)
    )
    (cv3): Conv(
        (conv): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
        (act): SiLU(inplace=True)
    )
    (cv4): Conv(
        (conv): Conv2d(512, 512, kernel_size=(1, 1), stride=(1, 1))
        (act): SiLU(inplace=True)
    )
    (m): ModuleList(
        (0): MaxPool2d(kernel_size=5, stride=1, padding=2, dilation=1, ceil_mode=False)
        (1): MaxPool2d(kernel_size=9, stride=1, padding=4, dilation=1, ceil_mode=False)
        (2): MaxPool2d(kernel_size=13, stride=1, padding=6, dilation=1, ceil_mode=False)
    )
    (cv5): Conv(
        (conv): Conv2d(2048, 512, kernel_size=(1, 1), stride=(1, 1))
        (act): SiLU(inplace=True)
    )
    (cv6): Conv(
        (conv): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
        (act): SiLU(inplace=True)
    )
    (cv7): Conv(
        (conv): Conv2d(1024, 512, kernel_size=(1, 1), stride=(1, 1))
        (act): SiLU(inplace=True)
    )
)

```

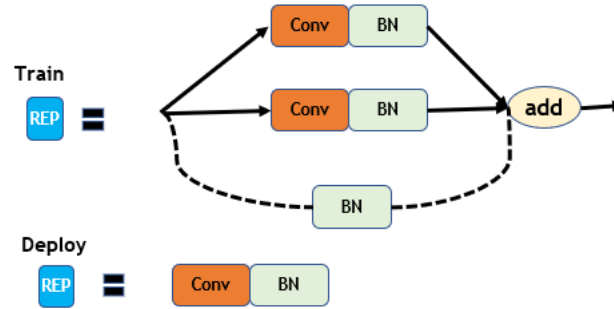
The function of SPP is to increase the receptive field, so that the algorithm can adapt to different resolution images. It obtains different receptive fields through maximum pooling . We can see that in the first branch, there are four branches that manage maxpool. They are 5,9, 13, and 1 respectively. These four different maxpools can handle different objects. That is to say, its maximum pooling of four different scales has four kinds of receptive fields, which are used to distinguish between large targets and small targets. For example, dogs, pedestrians, and cars in a photo have different scales. Through different maxpools, it is possible to better distinguish small targets from large targets. The CSP module first divides the features into two parts, one of which performs conventional processing, and the other part performs SPP structure processing, and finally merges these two parts together, which can reduce the amount of calculation by half leading to faster the speed and better the accuracy.

Interpretation of UPsample module

The UPsample module is an upsampling module, and the upsampling method it uses is nearest neighbor interpolation

(53): Upsample(scalefactor=2.0, mode=nearest)

Interpretation of REP module



```
(104): RepConv(
  (act): SiLU(inplace=True)
  (rbr_reparam): Conv2d(512, 1024, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
)
(105): Detect(
  (m): ModuleList(
    (0): Conv2d(256, 255, kernel_size=(1, 1), stride=(1, 1))
    (1): Conv2d(512, 255, kernel_size=(1, 1), stride=(1, 1))
    (2): Conv2d(1024, 255, kernel_size=(1, 1), stride=(1, 1))
  )
)
```

The REP module is divided into two, one is train, which is training, and the other is deploy

The training module has three branches. The top branch is a 3x3 convolution for feature extraction. The middle branch is a 1x1 convolution for smoothing features. The last branch is an Identity, which is moved directly without convolution operation. Finally add them together. In the training module, because the first layer is a 3x3 convolution, the second layer is a 1x1 convolution, and the last layer is an Identity. When the model is parameterized, it is necessary to convert the 1x1 convolution into a 3x3 convolution, and convert the Identity into a 3x3 convolution, and then perform an addition of a matrix, which is a matrix fusion process. Finally it adds weights to get a 3x3 convolution, that is to say, these three branches are fused into a line with

only one 3×3 convolution. Their weights are the result of the superposition of the three branches, and the matrix is also the result of the superposition of the three branches.

4.0.5 Results analysis

Following Graphs were captured as a part of metrics to measure the performance of the model



Roboflow Train Metrics



Validation Set **Test Set** Training Graphs

Average Precision by Class



Model Family	MAP	Speed CPU	Speed GPU	Fine Tuning	Small Object Detection
		YOLOv5 Nano			
YOLO V5	31	Nano P6	YOLO V5n	YOLOv5m (640 resolution)	Yes (1280 resolution)
YOLO V6	38	NA	YOLO V6n		YOLOv6l at 1280
YOLO V7	37	NA	YOLO V7		YOLOv7-W6



Roboflow Train Metrics



Validation Set Test Set Training Graphs

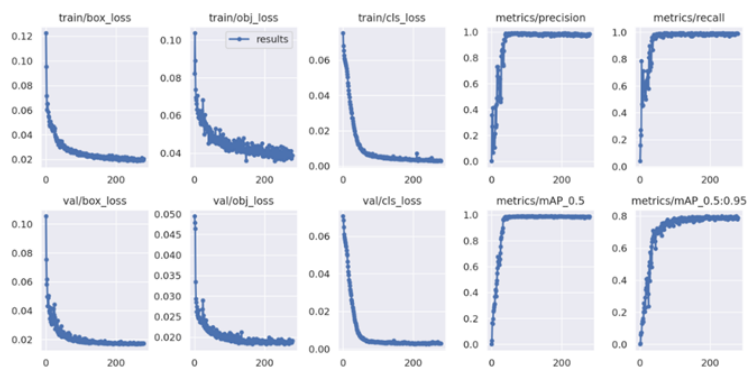
Average Precision by Class



Roboflow Train Metrics



Validation Set Test Set Training Graphs



Chapter 5

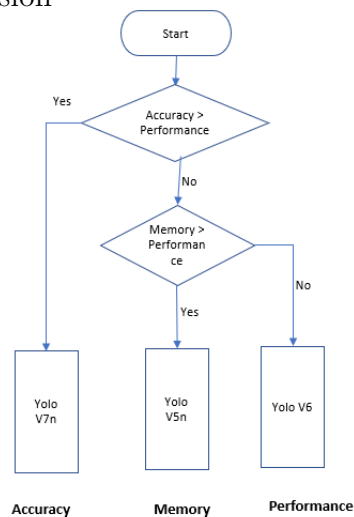
Conclusion and Future Directions

Computer vision has found wide range of applications in various domains ranging from automotive, medical, security and many others. A deep analysis using different YOLO version which is a state of art single shot detector found to be outperforming compared to multistage detectors. YOLO has been evolving rapidly to improve the performance parameters. This study aims to address key challenge in computer vision in terms of model performance with respect to accuracy, performance, and memory usage.

A comprehensive study was conducted on various Yolo architectures and the building blocks. As a part of this exercise, team also studied various applications developed using Yolo architecture in different domains such as object detection, pose estimation and motion detection. COCO dataset was used as reference. The study also included custom data consisting of chess pieces.

Team implemented and experimented with Yolo V5, V6 and V7. Experiment revealed that

1. V5 architecture is a stable and meeting the performance requirements. V5 is best suited version for Edge computing devices.
2. V6 has better capability of performance (FPS/FLOPS)
3. Yolo V7 provides highest accuracy measured through Mean Average Precision



YOLO can be applied in wide range of domains, some of the future directions are

- a) Optimizing Time Measurement in Manual Assembly using Key Point Extraction and Temporal Graph Neural Networks
- b) Unsafe condition Identification and Prevention using computer vision
- c) Abnormal Activity During Driving Detection using yolo
- d) Animal Detection and 3D world position estimation using YOLOv7 and FlowNet architectures
- e) Sustainable waste management using Object detection

Bibliography

- [1] Narayana Darapaneni, B Krishnamurthy, and Anwesh Reddy Paduri. Convolution neural networks: A comparative study for image classification. In *2020 IEEE 15th International Conference on Industrial and Information Systems (ICIIS)*, pages 327–332, 2020.
- [2] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 779–788, 2016.
- [3] Yichen Zhang, Jeong Hoon Han, Yong Woo Kwon, and Young Shik Moon. A new architecture of feature pyramid network for object detection. In *2020 IEEE 6th International Conference on Computer and Communications (ICCC)*, pages 1224–1228, 2020.
- [4] Joseph Redmon and Ali Farhadi. Yolov3: An incremental improvement. *CoRR*, abs/1804.02767, 2018.
- [5] Pranav Adarsh, Pratibha Rathi, and Manoj Kumar. Yolo v3-tiny: Object detection and recognition using one stage improved model. In *2020 6th International Conference on Advanced Computing and Communication Systems (ICACCS)*, pages 687–694, 2020.
- [6] Wei Fang, Lin Wang, and Peiming Ren. Tinier-yolo: A real-time object detection method for constrained environments. *IEEE Access*, 8:1935–1944, 2020.
- [7] Alexey Bochkovskiy, Chien-Yao Wang, and Hong-Yuan Mark Liao. Yolov4: Optimal speed and accuracy of object detection, 2020.
- [8] Omar Elharrouss, Younes Akbari, Noor Almaadeed, and Somaya Al-Maadeed. Backbones-review: Feature extraction networks for deep learning and deep reinforcement learning approaches, 2022.

- [9] Chien-Yao Wang, Hong-Yuan Mark Liao, Yueh-Hua Wu, Ping-Yang Chen, Jun-Wei Hsieh, and I-Hau Yeh. Cspnet: A new backbone that can enhance learning capability of cnn. In *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, pages 1571–1580, 2020.
- [10] Chien-Yao Wang, Hong-Yuan Mark Liao, I-Hau Yeh, Yueh-Hua Wu, Ping-Yang Chen, and Jun-Wei Hsieh. Cspnet: A new backbone that can enhance learning capability of cnn, 2019.
- [11] Gao Huang, Zhuang Liu, Laurens van der Maaten, and Kilian Q. Weinberger. Densely connected convolutional networks, 2016.
- [12] Joseph Redmon and Ali Farhadi. Yolov3: An incremental improvement, 2018.
- [13] Shu Liu, Lu Qi, Haifang Qin, Jianping Shi, and Jiaya Jia. Path aggregation network for instance segmentation. In *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 8759–8768, 2018.
- [14] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Spatial pyramid pooling in deep convolutional networks for visual recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 37(9):1904–1916, 2015.
- [15] Zicong Jiang, Liquan Zhao, Shuaiyang Li, and Yanfei Jia. Real-time object detection method based on improved yolov4-tiny. 2020.
- [16] Jie Liu and Lizhi Liu. Helmet wearing detection based on yolov4-mt. In *2021 4th International Conference on Robotics, Control and Automation Engineering (RCAE)*, pages 1–5, 2021.
- [17] Yuqiao Gai, Weiyang He, and Zilong Zhou. Pedestrian target tracking based on deepsort with yolov5. In *2021 2nd International Conference on Computer Engineering and Intelligent Control (ICCEIC)*, pages 1–5, 2021.
- [18] Glenn Jocher, Ayush Chaurasia, Alex Stoken, Jirka Borovec, NanoCode012, Yonghye Kwon, TaoXie, Kalen Michael, Jiacong Fang, imyhxy, Lorna, Colin Wong, (Zeng Yifu), Abhiram V, Diego Montes, Zhiqiang Wang, Cristi Fati, Jebastin Nadar, Laughing, UnglvKitDe, tkianai, yxNONG, Piotr Skalski, Adam Hogan, Max Strobel, Mrinal Jain, Lorenzo Mammana, and xylieong. ultralytics/yolov5: v6.2 -

YOLOv5 Classification Models, Apple M1, Reproducibility, ClearML and Deci.ai integrations, August 2022.

- [19] Aybora Köksal, Önder Tuzcuoğlu, Kutalmış Gökalp İnce, Yoldaş Ataseven, and A Aydın Alatan. Improved hard example mining approach for single shot object detectors. In *2022 IEEE International Conference on Image Processing (ICIP)*, pages 3536–3540. IEEE, 2022.
- [20] Shasha Li, Yongjun Li, Yao Li, Mengjun Li, and Xiaorong Xu. Yolo-firi: Improved yolov5 for infrared image object detection. *IEEE Access*, 9:141861–141875, 2021.
- [21] Chuyi Li, Lulu Li, Hongliang Jiang, Kaiheng Weng, Yifei Geng, Liang Li, Zaidan Ke, Qingyuan Li, Meng Cheng, Weiqiang Nie, et al. Yolov6: a single-stage object detection framework for industrial applications. *arXiv preprint arXiv:2209.02976*, 2022.
- [22] Haoyang Zhang, Ying Wang, Feras Dayoub, and Niko Sunderhauf. Varifocalnet: An iou-aware dense object detector. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 8514–8523, 2021.
- [23] Linfeng Zhang, Jiebo Song, Anni Gao, Jingwei Chen, Chenglong Bao, and Kaisheng Ma. Be your own teacher: Improve the performance of convolutional neural networks via self distillation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 3713–3722, 2019.
- [24] Chien-Yao Wang, Alexey Bochkovskiy, and Hong-Yuan Mark Liao. Yolov7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors, 2022.
- [25] Kailin Jiang, Tianyu Xie, Rui Yan, Xi Wen, Danyang Li, Hongbo Jiang, Ning Jiang, Ling Feng, Xuliang Duan, and Jianjun Wang. An attention mechanism-improved yolov7 object detection algorithm for hemp duck count estimation. *Agriculture*, 12(10), 2022.
- [26] Xiaohan Ding, Xiangyu Zhang, Ningning Ma, Jungong Han, Guiguang Ding, and Jian Sun. Repvgg: Making vgg-style convnets great again. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 13733–13742, 2021.

- [27] H.A.M Priyanwada, K.A.D Dilanka Madhushan, Chethana Liyanapathirana, and Lakmal Rupasinghe. Vision based intelligent shelf-management system. In *2021 6th International Conference on Information Technology Research (ICITR)*, pages 1–6, 2021.
- [28] Ahmet Furkan Bayram, Caglar Gurkan, Abdulkadir Budak, and Hakan KARATAŞ. A detection and prediction model based on deep learning assisted by explainable artificial intelligence for kidney diseases. *Avrupa Bilim ve Teknoloji Dergisi*, (40):67–74, 2022.
- [29] Kwang-Ju Kim, Pyong-Kun Kim, Yun-Su Chung, and Doo-Hyun Choi. Multi-scale detector for accurate vehicle detection in traffic surveillance data. *IEEE Access*, 7:78311–78319, 2019.
- [30] Mingxin Jiang, Tao Hai, Zhigeng Pan, Haiyan Wang, Yinjie Jia, and Chao Deng. Multi-agent deep reinforcement learning for multi-object tracker. *IEEE Access*, 7:32400–32407, 2019.
- [31] Shaobin Chen and Wei Lin. Embedded system real-time vehicle detection based on improved yolo network. In *2019 IEEE 3rd Advanced Information Management, Communicates, Electronic and Automation Control Conference (IMCEC)*, pages 1400–1403, 2019.
- [32] Ping Ping, Xiaohui Yang, and Zeyu Gao. A deep learning approach for street pothole detection. In *2020 IEEE Sixth International Conference on Big Data Computing Service and Applications (BigDataService)*, pages 198–204, 2020.
- [33] Jia Kang, Wenjun Liu, Wenzhe Tu, and Lu Yang. Yolo-6d+: Single shot 6d pose estimation using privileged silhouette information. In *2020 International Conference on Image Processing and Robotics (ICIP)*, pages 1–6, 2020.
- [34] Yao Zhang, Zhiyong Chen, and Bohan Wei. A sport athlete object tracking based on deep sort and yolo v4 in case of camera movement. In *2020 IEEE 6th International Conference on Computer and Communications (ICCC)*, pages 1312–1316. IEEE, 2020.
- [35] Akansha Bathija and Grishma Sharma. Visual object detection and tracking using yolo and sort. *International Journal of Engineering Research Technology*, 8(11), 2019.
- [36] Wenbo Lan, Jianwu Dang, Yangping Wang, and Song Wang. Pedestrian detection based on yolo network model. In *2018 IEEE International*

- Conference on Mechatronics and Automation (ICMA)*, pages 1547–1551, 2018.
- [37] Shubham Shinde, Ashwin Kothari, and Vikram Gupta. Yolo based human action recognition and localization. *Procedia computer science*, 133:831–838, 2018.
 - [38] Cheng Yan, Xin Li, and Guoqiang Li. A new action recognition framework for video highlights summarization in sporting events. In *2021 16th International Conference on Computer Science Education (ICCSE)*, pages 653–666, 2021.
 - [39] Zhe Cao, Gines Hidalgo, Tomas Simon, Shih-En Wei, and Yaser Sheikh. Openpose: Realtime multi-person 2d pose estimation using part affinity fields. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 43(1):172–186, 2021.
 - [40] Jun Wu, Yu Li, Liuqing Wang, Ke Wang, Ruifeng Li, and Tianxiang Zhou. Skeleton based temporal action detection with yolo. *Journal of Physics: Conference Series*, 1237(2):022087, jun 2019.
 - [41] Chunfang Liu, Xiaoli Li, Qing Li, Yaxin Xue, Huijun Liu, and Yize Gao. Robot recognizing humans intention and interacting with humans based on a multi-task model combining st-gcn-lstm model and yolo model. *Neurocomputing*, 430:174–184, 2021.
 - [42] Yefan Zhou, Zhao Lv, and Yongtai Zhou. Ddgnet-yolo: A target detection network for dangerous driving gestures. In *2020 Chinese Automation Congress (CAC)*, pages 3047–3052, 2020.
 - [43] May Thu Soe, Aung Zaw Min, Hayma Thida Kyaw, Myo Min Paing, Sai Myo Htet, and Bawin Aye. Abnormal behavior detection in real-time for advanced driver assistance system (adas) using yolo. In *2022 IEEE Symposium on Industrial Electronics Applications (ISIEA)*, pages 1–6, 2022.
 - [44] Yogesh R, Vaddi Ritheesh, Suneel Reddy, and Rajesh George Rajan. Driver drowsiness detection and alert system using yolo. In *2022 International Conference on Innovative Computing, Intelligent Communication and Smart Electrical Systems (ICSES)*, pages 1–6, 2022.
 - [45] Yi Sun, Xiaogang Wang, and Xiaoou Tang. Deep convolutional network cascade for facial point detection. In *2013 IEEE Conference on Computer Vision and Pattern Recognition*, pages 3476–3483, 2013.

- [46] Ali Raza, Muhammad Haroon Yousaf, and Sergio A Velastin. Human fall detection using yolo: A real-time and ai-on-the-edge perspective. In *2022 12th International Conference on Pattern Recognition Systems (ICPRS)*, pages 1–6. IEEE, 2022.
- [47] Deng Benyang, Lei Xiaochun, and Ye Miao. Safety helmet detection method based on yolo v4. In *2020 16th International Conference on Computational Intelligence and Security (CIS)*, pages 155–158. IEEE, 2020.
- [48] Ahmed Abobakr, Hala Abdelkader, Julie Iskander, Darius Nahavandi, Khaled Saleh, Mohamed Attia, Mohammed Hossny, and Saeid Nahavandi. Ssdpose: A single shot deep pose estimation and analysis. In *2019 IEEE International Conference on Systems, Man and Cybernetics (SMC)*, pages 1862–1868. IEEE, 2019.
- [49] G Andrew, Zhu Menglong, et al. Efficient convolutional neural networks for mobile vision applications. *Mobilenets*, 2017.
- [50] Günter Klambauer, Thomas Unterthiner, Andreas Mayr, and Sepp Hochreiter. Self-normalizing neural networks. *Advances in neural information processing systems*, 30, 2017.
- [51] Prajit Ramachandran, Barret Zoph, and Quoc V Le. Swish: a self-gated activation function. *arXiv preprint arXiv:1710.05941*, 7(1):5, 2017.
- [52] Diganta Misra. Mish: A self regularized non-monotonic neural activation function. *arXiv preprint arXiv:1908.08681*, 4(2):10–48550, 2019.
- [53] P. Bruce and A. Bruce. *Practical Statistics for Data Scientists: 50 Essential Concepts*. O’Reilly Media, 2017.
- [54] M. Elgendy. *Deep Learning for Vision Systems*. Manning Publications, 2020.
- [55] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection, 2017.
- [56] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization, 2014.
- [57] John Duchi, Elad Hazan, and Yoram Singer. Adaptive subgradient methods for online learning and stochastic optimization. *Journal of Machine Learning Research*, 12(61):2121–2159, 2011.

- [58] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick. Microsoft coco: Common objects in context. In *European conference on computer vision*, pages 740–755. Springer, 2014.